

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a67e419dad1b493da.jsonl`  
- SHA-256: `e6295d4f70cd30b70e5747f391e5e9c54cf730f5707f62abe9c0782a3fa48bf5`  
- Source modified: `2026-06-10T05:44:08+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T05:33:14.936Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

TASK: full audit (engineering + testing + reliability + docs) of the COLLECTOR repo at C:/Users/avidu/Projects/sports-data-collector.

Steps: read README.md and all of docs/ (DECISIONS.md, PIPELINE_ROLE.md, INGESTION_PIPELINE.md, MANUAL_INGESTION_POSITIONING.md, annotation/*). Map the src/ tree (Glob src/**/*.py) and deep-read the core: src/cli/curate.py, src/cli/normalize_annotations.py, src/parsers/ (esp. cvat/rally_cvat.py), the db layer, models/validation, reporting/. RUN the test suite (pytest from repo root ? offline, safe); capture real counts.

ASSESS: (1) engineering quality and architecture vs its stated role as the SYSTEM-OF-RECORD for the training corpus (docs/MANUAL_INGESTION_POSITIONING.md) ? is it actually ready for that role? what's missing (provenance, maturity ladder, labeled_window persistence ? check which of these exist TODAY vs are documented as P1/P2); (2) known landmines: import-local DELETE-then-INSERT destructiveness, int(float()) rally_number PK behavior, pydantic abort-whole-import semantics, the normalize_annotations guard coverage; (3) the indexer->collector CONTRACT: does "curate export" output (manifest.json + per-video start,end CSV) actually match what the indexer's backend/eval/batch.py + run_phase3_eval.py consume (read those indexer files to cross-check field names/paths/Windows path normalization)? does convert-cvat's ATTR_MAP handle the real Roora CVAT attribute names (spaces, "(A-B)")? (4) test coverage gaps and quality; (5) reliability: what happens on partial/corrupt CVAT zips, duplicate imports, fps mismatch. Severity-rate as if the training corpus' integrity is on the line (it is ? this feeds the owned model).

2. user (2026-06-10T05:33:21.275Z)

```
README.md  
__pycache__
```

```
cleanup.py
config
data
docs
pyproject.toml
src
tests
```

Sports Data Collector & Curator

A clean, extensible Python codebase and CLI toolchain designed to discover, ingest, validate, and curate sports video datasets. The repository stores annotations and video metadata in an SQLite database, supports sport-specific tables and schemas, manages video storage outside of Git, and enforces strict commercial license compliance.

> **Pipeline role:** this repo is the *data producer* feeding the *badminton-highlight-indexer*, which trains/evaluates an offline rally segmenter on this curated, commercially-licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist and how the two repos fit together.

Features

- **SQLite Backend:** Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).
- **Commercial License Flagging:** Each ingested source license is checked against a configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`, `Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ? nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently across the parser, validator, and CLI.
- **Overlap & Duration Validation:** Ensures chronological event bounds (e.g. rally start/end times) do not overlap and that durations are positive.
- **Stroke-level CSV Parser:** Ingests ShuttleSet/CoachAI-style stroke CSV logs and aggregates them into rally durations using configurable start/end time buffers.
- **Local Ingestion Mode:** Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.
- **Interactive Curation CLI & Visual Play Harness:** Let's you query DB status, list staging entries, review sample timestamps, and approve/reject them. It includes a built-in play function that opens the video in your default browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files or YouTube start time markers `&t=start` for remote videos) for manual visual verification.

Installation

Ensure you have Python 3.9+ installed.

1. Clone or download the repository.
2. Install the package and dependencies:

```
```bash
pip install -e . # core: curate / import-local / export / status
pip install -e ".[fetch]" # adds yt-dlp for video acquisition (fetch / crawler)
pip install -e ".[dev]" # test tooling (pytest, pytest-cov)
```
```

External requirements (not pip-installable)

The `fetch` command and the ShuttleSet crawler shell out to external binaries. They degrade gracefully (a clear error / skipped download) when missing, but are **required** for video acquisition:

- **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
- **A JS runtime (e.g. [deno](https://deno.com))** ? needed by yt-dlp to solve YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno install at `~/deno/bin`.

Core curation/import/export/status commands work without any of these.

Usage

All commands should be executed from the root of the project. Make sure `PYTHONPATH` is set to the current directory:

```
```bash
```

## On Windows PowerShell:

```
$env:PYTHONPATH="."
```

```
```
```

1. View Database Status

To check the current statistics of staging vs. curated items:

```
```bash
```

```
python -m src.cli.curate status
```

```
```
```

2. Ingest External Datasets to Staging

To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in staging:

```
```bash
```

```
python -m src.cli.ingest --sport badminton --parser coachai --file-path
data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
```

```
```
```

3. Launch the Interactive Curation Tool & Play Harness

To review staging items, inspect their timestamps, play individual rallies in the browser to visually verify their start and end bounds, and approve or reject them:

```
```bash
```

```
python -m src.cli.curate curate
```

```
```
```

4. Manually Import Local Videos & Annotations

To register a local video file alongside its JSON/CSV annotation file (bypasses staging):

```
```bash
```

```
python -m src.cli.curate import-local --sport badminton --video-path
"data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json"
--match-name "My Custom Local Match" --license Proprietary
```

```
```
```

`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`, `pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g. via an annotation vendor), see [`docs/annotation/`](docs/annotation/).`

5. Fetch / Upgrade Videos to Best Resolution

Videos are **always** downloaded at the best resolution available (via yt-dlp `bv*+ba/b -S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed. The `fetch` command (re)downloads videos for entries already in the DB and updates `local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation status (no clean slate needed):

```
```bash
```

## Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies

```
python -m src.cli.curate fetch --cookies-from-browser firefox
```

## A single video, forcing a re-download even if a good local copy exists:

```
python -m src.cli.curate fetch --video-id shuttlest_1 --force --cookies-from-browser firefox
```


Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`), `--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height` (cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`, `--cookies-from-browser/--cookies`, `--player-client`. A global resolution cap can also be set via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a download that falls below `--min-height` is treated as a failure (discarded, DB left unchanged) so a flaky low-res result never overwrites a good entry.


```

```
> Authenticated cookies are strongly recommended. Anonymous YouTube downloads are throttled
and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser firefox` (or
`chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a logged-in
(ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-bound
cookie encryption may block direct reads on Windows; Firefox reads cleanly even while open.
>
> JS runtime: yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's challenges
for high-res formats; `fetch` auto-detects a deno install at `~/deno/bin`. The default
`--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming (drops to
144p).
```

6. Export Annotations as Indexer-Ready Eval/Training Sets

To turn curated annotations into ground-truth files the badminton-highlight-indexer can score and train against, export per-video `start,end` CSVs plus a `manifest.json`:

```
```bash
```

### Curated + commercially-safe videos across all supported sports (default):

```
python -m src.cli.curate export --out-dir data/eval_export
```

### One sport, also including staging entries:

```
python -m src.cli.curate export --sport badminton --include-staging
```
```

The exporter writes `//.csv` (decimal-second `start,end` rows) and a `manifest.json` pairing each CSV with its video `local\_path`/`video\_url`, fps, resolution, and license. The indexer keys videos by **file MD5**, not by our `video\_id`, so the downstream eval/training run must process the exact video file named in the manifest. Flags: `--sport` (restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial` (default: commercial-safe only).

```
---
```

Configuration

Adjust settings in `config/config.yaml`:

- **database_path**: Path to the SQLite database.
- **video_storage**: Local folder path and automatic download size limit.
- **commercial_licenses**: Whitelist of licenses allowed for commercial use.
- **supported_sports**: Allowed sports categories.

```
---
```

Documentation

- docs/PIPELINE_ROLE.md ? how this repo feeds the **badminton-highlight-indexer** (`export` / `fetch`).
- docs/DECISIONS.md ? collector-local decision records (**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport" checklist).
- docs/annotation/ ? the vendor annotation package used to produce golden rally labels:
 - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical rally CSV schema.
 - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater guide (singles & doubles).

- [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-video intake sheet.
- [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have counsel review).

Running Tests

We use `pytest` for unit and integration testing:

```
```bash
python -m pytest tests/
python -m pytest tests/ --cov=src --cov-report=term-missing # with coverage (~87%)
```
```

3. user (2026-06-10T05:33:22.070Z)

```
src\core\schemas\tennis.py
src\core\schemas\cricket.py
src\parsers\base_parser.py
src\core\downloader.py
src\cli\ingest.py
src\core\config.py
src\core\parsing.py
src\core\schemas\base.py
src\core\validator.py
src\crawlers\badminton_crawler.py
src\parsers\badminton\coachai.py
src\parsers\cvat\__init__.py
src\core\db.py
src\core\schemas\badminton.py
src\core\schemas\pickleball.py
src\core\schemas\tabletennis.py
src\parsers\cvat\rally_cvat.py
src\cli\curate.py
src\reporting\__init__.py
src\reporting\adapter.py
src\cli\normalize_annotations.py
```

4. user (2026-06-10T05:33:22.660Z)

```
docs\annotation\INTAKE_MANIFEST_TEMPLATE.csv
docs\annotation\NDA_TEMPLATE.md
docs\DECISIONS.md
docs\PIPELINE_ROLE.md
docs\annotation\ANNOTATION_GUIDE.md
docs\annotation\ROORA_BRIEF.md
docs\INGESTION_PIPELINE.md
docs\MANUAL_INGESTION_POSITIONING.md
```

5. user (2026-06-10T05:33:28.165Z)

```
1      # Decision records
2
3      Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in
4      the indexer repo's `docs/DECISIONS.md`; this file is for collector-local choices.)
5
6      ---
7
8      ## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
9
10     **Status:** Accepted . **Date:** 2026-06-02
11
12     ### Context
```

```

13     Supporting a sport currently requires parallel edits across several sites:
14     - `SEGMENT_TABLES` + a `CREATE TABLE` + `insert_*` / `get_*` in `src/core/db.py`
15     - a `validate_*` method in `src/core/validator.py`
16     - `import-local` parse/validate/insert branches and interactive review/play in
`src/cli/curate.py`
17     - a schema in `src/core/schemas/`
18
19     A code review flagged this as copy-paste duplication and proposed a registry/plugin
20     mechanism (e.g. `SPORT_REGISTRY: dict[SportType, SportSpec]` driving generic
21     insert/get/validate/dispatch) to make adding a sport a single entry.
22
23     ### Decision
24     **We are keeping the explicit per-sport code and NOT introducing a registry
25     abstraction at this time.**
26
27     ### Rationale
28     - **It's well-tested and mechanical.** Coverage is ~87%; adding a sport is a known,
29       ~30-minute "mirror an existing sport" task (checklist below), not a research problem.
30     - **Schemas are heterogeneous.** Rally sports (badminton, pickleball, table tennis)
31       share `rally_number` / `shots_count` / `ending_reason`, but tennis (`set_score`,
32       `game_score`, `server`, `ending_type`) and cricket (`over`, `ball`, `runs`,
33       `wicket`, `extras`) differ enough that a single generic table/row mapper would leak
34       special-cases ? trading visible duplication for hidden complexity.
35     - **Explicit code is greppable.** Contributors can follow one sport end-to-end without
36       learning a registry indirection. Premature abstraction is harder to undo than dup.
37     - **Low, slow growth.** Five sports today; new ones arrive rarely.
38
39     ### Consequences
40     - Adding a sport touches ~5 sites (see checklist). We accept that boilerplate.
41     - If the cost rises, ADR-001 should be revisited and superseded.
42
43     ### Revisit triggers (when to build the registry)
44     - Sport count grows beyond ~8, **or**
45     - We add a 4th+ rally-identical sport and the dispatch churn starts causing bugs, **or**
46     - A new cross-cutting feature would otherwise require editing every per-sport branch.
47
48     ### Future design sketch (when revisited)
49     ```
50     @dataclass
51     class SportSpec:
52         schema_class: type
53         table_name: str
54         pk_columns: tuple[str, ...]
55         columns: tuple[str, ...]
56         validate_fn: Callable
57
58     SPORT_REGISTRY: dict[SportType, SportSpec] = { ... }
59     # Generic: insert_segments(sport, items) / get_segments(sport, video_id) /
60     #       validate(sport, items) / import-local dispatch driven by the registry.
61     ```
62
63     ---
64
65     ## Checklist: adding a new sport today (under ADR-001)
66
67     1. `src/core/schemas/<sport>.py` ? define the segment model (mirror the closest existing
sport) with a `start_time < end_time` validator.
68     2. `src/core/db.py` ? add to `SEGMENT_TABLES`, add a `CREATE TABLE`, `insert_<sport>()`,
`get_<sport>()`, and a cascade `DELETE` in `delete_video`.
69     3. `src/core/validator.py` ? add `validate_<sport>()` (duration + no-overlap).
70     4. `src/cli/curate.py` ? add branches in `import_local` (validate + insert),
`_parse_local_annotations` (JSON **and** CSV), and the interactive review/play.
71     5. `config/config.yaml` ? add the sport to `supported_sports` (and confirm the
`SportType` enum value in `schemas/base.py`).
72     6. `tests/` ? mirror the existing db/validator/cli tests for the new sport.

```

6. user (2026-06-10T05:33:28.628Z)

```

1      # Role in the rally-segmenter training pipeline
2
3      This repo is the **data producer** in a two-repo system. Its job is to curate
4      sports videos with per-rally `[start, end)` annotations and commercial-license
5      metadata, then hand that data to the **badminton-highlight-indexer**, which
6      processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.
7
8      This doc explains *why* the `export` and `fetch` commands exist and how they fit
9      the bigger picture. The full decision records and end-to-end narrative live in the
10     indexer repo:
11
12     - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13     - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14       (acquisition policy), ADR-004 (offline learned segmenter).
15
16     ## Why this repo matters
17
18     The indexer's best rally mode refines local detections with a **paid Gemini API**.
19     The strategic goal is to replace that with an **offline model trained on our own
20     commercially-licensed, annotated corpus** ? making rally detection free, offline,
21     and self-improving. None of that is possible without a clean, licensable,
22     accurately-labelled, full-resolution dataset. That dataset is what this repo
23     produces.
24
25     ## What the two key commands do (and why)
26
27     - **`curate export`** ? emits the indexer's ground-truth format: per-video
28       `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29       Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30       table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31       `is_commercial`, so only licensable data flows
32       downstream. The annotations are **temporal-only** ? they train/evaluate rally
33       *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35     - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36       the DB **in place** so annotations and curation status are never lost. Uses
37       authenticated cookies for reliable, unthrottled downloads. The why and the
38       hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40     ## The one contract that matters: video identity
41
42     The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43     re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44     exact `local_path`, and any downstream processing must run on **that** file. This is
45     why acquisition (`fetch`) happens *before* the indexer processes anything.
46

```

7. user (2026-06-10T05:33:29.568Z)

```

1      # Rally Ingestion Pipeline ? End-to-End FAQ
2
3      How a vendor's rally annotations become training/eval ground truth, and how we
4      validate each delivery against our conventions. This is the operational companion
5      to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7      ...
8
9      vendor delivery                                OUR side (collector)                                modeling
(indexer)
10     ?????????????????? convert ?????????????????? validate ??????????????????
11     ? CVAT export ? ?????????????? ? canonical CSV ? ?????????????? ? report .md ?
12     ? (boxes + attrs)? ? (per rally, ? ? report .json ?

```

```

12      ? OR      ?      ? decimal secs)?      ? review .csv ??? human
verifies
13      ? canonical CSV ?      ??????????????????      ??????????????????
flagged rallies
14      ??????????????????      ?      ?
15      ? import-local (validated) ?
16      ?      golden labels
17      ?????????????????? export ??????????????????
18      ? SQLite (rally ? ?????????? ? <id>.csv      ?
start,end (indexer GT)
19      ? tables)      ?      ? manifest.json?
20      ??????????????????      ??????????????????
21      ?
22      slice clips / calibrate /
train
23      ```
24
25      ## The four commands (in order)
26
27      ```bash
28      # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29      python -m src.cli.curate convert-cvat --input job.xml \
30      --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32      # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33      python -m src.cli.curate validate-delivery --input job.xml \
34      --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35      # ...or validate a CSV directly:
36      python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38      # 3. After human review, import the (corrected) CSV into the DB
39      python -m src.cli.curate import-local --sport badminton \
40      --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42      # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43      python -m src.cli.curate export --out-dir data/eval_export
44      ```
45
46      ---
47
48      ## FAQ
49
50      ### What delivery formats do we accept?
51      Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52      attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53      as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54      CSVs.
55
56      ### What is the "canonical" CSV?
57      One row per rally:
58      `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
59      `start_time`/`end_time` are decimal seconds; `import-local` reads those.
60
61      ### How are timestamps handled? (the fps thing)
62      We recompute each rally's interval from its frame range × the true fps
63      (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
64      step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
65      from `--fps` or fprobe of `--video-path`. `mm:ss` is also accepted everywhere.
66
67      ### What does validation check?
68      | code | severity | meaning |
69      |---|---|---|

```



```

70 | `overlap` | **blocker** | two rallies' intervals overlap (`end[N] > start[N+1]`) |
71 | `zero_duration` | **blocker** | `end <= start` (single-sample / stray box) |
72 | `frame_out_of_range` | high | a box frame is past the video's last frame |
73 | `shot_density` | medium | long rally with implausibly few shots ? likely an **inflated
span** (often hides a merged rally) |
74 | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an **omitted rally** |
75 | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
76 | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
77
78 ### Blocker vs. review ? what's the gate?
79 `validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk
80 loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet**
81 for the human pass. `import-local` independently rejects overlaps/zero-duration, so
82 bad ground truth can't slip into the DB.
83
84 ### How are *missed rallies* caught? (the worst error)
85 Two complementary checks, because a miss shows up two ways:
86 1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap
heuristic).
87 2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
88 no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
89 the second rally inside it.
90 Neither is a guarantee ? they **surface regions to review**, they don't certify
91 completeness. (A future enhancement: a motion second-opinion over the raw video.)
92
93 ### What's the review sheet?
94 `<id>_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
95 blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
96 `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
97 (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98 a delivery into trusted golden labels.
99
100 ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101 They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102 unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103 last shot went (in / net / out / n_a). A shuttle that flew long is
104 `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106 ### How do I onboard a brand-new video?
107 1. Get fps into the intake manifest (we provide it to the vendor).
108 2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109 sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110 3. The same suite runs for every video, so quality stays consistent at scale.
111
112 ### How do I pull a single rally out to eyeball it?
113 The slicer (indexer):
114 ```bash
115 python -m backend.tools.rally_slicer --video match.mp4 \
116 --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117 ```
118 Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120 ### Does coarse frame sampling hurt boundaries?
121 Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122 padding, so we don't flag sampling density.
123
124 ### Will a non-commercial license block export?
125 The video is **kept** in the DB regardless of license, but the default `export` is
126 **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127 from the eval/training set unless you pass `--include-noncommercial`. So if a video
128 "goes missing" downstream, check its `is_commercial` flag first. (The observability
129 report raises a `license_noncommercial` warning for exactly this case.)
130
131 ### What is the observability report? (`*.report.json` / `*.report.md` / `*.summary.md`)

```

```

132 Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133 per-video observability report (next to the video for `import-local`; into the
134 `--out-dir` / next to the canonical CSV otherwise):
135 - <id>.report.json ? machine-readable: every stage, metric, and finding. Diff
136 two runs to see what changed between deliveries.
137 - <id>.report.md ? the technical report: parse / validation / registration
138 stages plus findings (one per delivery issue, each citing this FAQ). Read this
139 to debug a delivery.
140 - <id>.summary.md ? a delivery summary for the vendor / data team: the
141 rally count and coverage plus the plain-language list of rallies to review ? no
142 internal codes or severities.
143 This is in addition to the existing <id>_report.md / _report.json / _review.csv
144 (the human review sheet), which are unchanged.
145

```

8. user (2026-06-10T05:33:29.784Z)

```

1  # Positioning: manual (rally-annotator) golden-label ingestion
2
3  How `sports-data-collector` should be positioned to ingest manual rally labels
4  produced by the VLC [rally-annotator](https://github.com/avidullu/rally-annotator),
5  alongside vendor (Roora/CVAT) deliveries, and serve them to the
6  badminton-highlight-indexer eval/training pipeline.
7
8  > Source: a 10-agent analysis workflow across this repo, the indexer, and the
9  > annotator, verified against the real `Adarsh and Avi.rallies.csv`. Confidence: high.
10
11  ## Bottom line
12
13  Position the collector as the single canonical system-of-record for golden rally
14  labels: the one place where any annotation source ? manual VLC lean CSV, the rich
15  review sheet, or vendor CVAT/CSV ? becomes a validated, license-gated, registered rally
16  record. The indexer consumes only the collector's narrow export contract:
17  `manifest.json` + a per-video `start,end` CSV (`curate.py::export_eval_set`;
18  `calibrate_wasb.py::load_golden_gts` reads `start_time`/`end_time` only).
19
20  The load-bearing property to protect: the seam to the indexer stays intervals-only.
21  Everything else ? `ending_reason`, `shots_count`, scores, provenance, maturity, the
22  labeled window ? is validated at the hub and rides in the manifest, never the eval
23  CSV. A new sport or source becomes an adapter change at the hub, never an indexer
24  change.
25
26  ## Role in the pipeline
27
28  producers ?????????????? collector (hub) ?????????????? consumer
29  ? VLC rally-annotator      parse ? validate ? license-gate  badminton-highlight-
30  (lean 5-col CSV)           ? register (SQLite) ? export      indexer (eval/training)
31  ? manual review sheet                                     reads ONLY:
32  (rich 14-col CSV)         everything but (start,end) is      manifest.json +
33  ? vendor Roora / CVAT     validated then dropped before     per-video start,end
34  CSV
35  (CVAT XML / canonical)    the indexer sees it
36  ...
37
38  ## It already works today (zero code changes)
39
40  The lean annotator CSV header `rally_number,start_time,end_time,ending_reason,sport`
41  matches the `import-local` `DictReader` exactly (`curate.py::_parse_local_annotations`),
42  and the validator accepts gapped/partial coverage as-is
43  (`validator.py::validate_badminton_rallies` checks only positive duration + no overlap).
44  Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.
45
46  ```bash
47  python -m src.cli.curate import-local --sport badminton \

```

```

47     --video-path "?/Adarsh and Avi.MP4" \
48     --annotations-path "?/Adarsh and Avi.rallies.csv" --license Proprietary --fps 59.94
49 python -m src.cli.curate export # ? manifest.json + <video_id>.csv (start,end)
50 ```
51
52 ## P0 ? the two-landmine guard (`normalize_annotations.py`, shipped)
53
54 Partial human data has two failure modes the raw path handles badly. The
55 `src/cli/normalize_annotations.py` pre-import normalizer fixes both, as a **standalone
56 step** that feeds `import-local` (it does **not** touch the DB schema or the
57 `rally_number`-ordered export):
58
59 1. **An unusable time row aborts the *entire* import.** A blank/missing time makes
60 `safe_float_required` raise (`parsing.py`); a reversed `end_time <= start_time` or a
61 negative `start_time` makes the `BadmintonRally` pydantic validator raise
62 (`schemas/badminton.py::validate_timestamps`). Either way the `try/except` in
63 `import-local` turns it into `sys.exit(1)`, killing the whole video. ? the normalizer
64 **drops** such rows (mirroring the indexer loader, which already skips `not end >
65 start`).
66 2. **A fractional/duplicate `rally_number` (e.g. `7.5`, `16.5`) silently corrupts
67 data.**
68     `int(float("7.5"))` ? 7` (`parsing.py::safe_int`) collides on the
69     `(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert
70     (`db.py::insert_badminton_rallies`) destroys the colliding rally. ? the normalizer
71     **renumbers `1..N` by `start_time`**, recording the original id for traceability.
72
73 It also converts `mm:ss`/`HH:MM:SS` ? decimal seconds and folds a pipe-delimited
74 `ending_reason` ("forced_error|other") to its primary token.
75
76 ```bash
77 python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
78 python -m src.cli.curate import-local --sport badminton --video-path "?/Match.MP4" \
79     --annotations-path clean.csv --license Proprietary
80 ```
81
82 **Do not widen the PK to float** ? the DB column is INTEGER, export orders by it, and
83 the indexer ignores `rally_number` entirely. Resolve ids at the adapter boundary.
84
85 ## Annotator ? canonical field mapping
86
87 | Annotator field | Canonical | Handling |
88 |---|---|---|
89 | `rally_number` (lean) / `rally` (rich) | `rally_number:int` | Lean maps directly;
90 **fractional/dup ids renumbered at the normalizer** (PK is INT) |
91 | `start_time` / `start_mmss` | `start_time:float` | Decimal direct; `mm:ss` converted
92 to seconds |
93 | `end_time` / `end_mmss` | `end_time:float` | **Blank / ? start dropped, not raised** |
94 | `ending_reason` | `ending_reason:str` | Vocab matches the documented canonical set
95 (free-text, not an enforced enum); pipe-delimited folded to primary; **stored but dropped on
96 export** |
97 | `sport` | ? (from `--sport` flag) | CSV column ignored; operator sets the flag |
98 | rich-only `shots`, `true_*`, `VERIFIED`, `notes` | `shots_count` + QA/provenance |
99 Carried where present; none reach the indexer |
100
101 ## Roadmap to the canonical-hub end-state
102
103 All additive, video-level, backward-compatible (consumers ignore unknown manifest keys).
104
105 - **P1 ? persist the labeled window in the manifest.** Partial labels cover only a
106 prefix; a full-video eval penalizes correct detections in the unlabeled tail. The
107 indexer already restricts to a window (`eval_partial_labels.py::restrict_to_window`)
108 but currently *infers* `window_end = max(end_time)` ? which is **wrong when a video is
109 tagged past its last rally** (valid predictions in the labeled-but-rally-free tail get
110 scored as false positives). The collector should own `labeled_window_start/end`
111 (the VLC tool is resumable + monotonic, so the session extent is known), carry it in

```

```

105     the manifest, and let the indexer apply it automatically. Short-term: pass
106     `--window-end` at eval time.
107 - **P1 ? maturity ladder + provenance.** `CurationStatus` (STAGING/CURATED/REJECTED) is
108   a *workflow* state, not a *trust* state. Add a separate `needs_review ? confirmed ?
109   gold` enum + `annotation_source` (manual_vlc / manual_review / vendor_cvat) +
110   `annotator_id` + timestamp. Route self-rated imports to `needs_review` instead of the
111   hardcoded `CURATED` (`curate.py`), so the golden-set regression flow restricts the
gold
112   tier to `maturity=gold` and an unreviewed re-tag can't silently become the answer key.
113   (Today a single-rater self-rating and a vendor-QA'd delivery are indistinguishable.)
114 - **P2 ? unify sources behind one adapter** emitting `(List[RallyRecord], List[Issue])`,
115   and run the **same Issue detectors** (gap>15s possible-missed-rally, shot-density,
116   vocab, overlap) on manual imports that vendor CVAT already gets. Today self-produced
117   labels get *less* scrutiny than a vendor delivery ? backwards.
118 - **P2 ? freeze the intervals-only export contract** in one doc both repos reference +
119   a round-trip test (header matching is exact case-insensitive, so a vendor
120   `start_seconds` would fail silently).
121
122   ## Open question
123
124   Is the **rich** `rally_review.csv` (mm:ss times, a `rally` column with fractional ids,
125   pipe-delimited reasons, `true_` correction columns) ever the *source of truth*, or is
126   the lean VLC `rallies.csv` always canonical? Neither `import-local` nor
127   `load_canonical_csv` (which keys on `rally_number`) ingest the rich file directly. The
128   `normalize_annotations.py` step already handles its `mm:ss` + `rally` + pipe-reason
129   quirks for the common case; a fuller converter that applies `true_` overrides is only
130   worth building if the rich sheet is canonical.
131

```

9. user (2026-06-10T05:33:34.815Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:

```

```

34         if not os.path.exists(self.config_path):
35             print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36         return load_config(self.config_path)
37
38     def status(self):
39         """Displays status summary of all videos in the database."""
40         # Query total count by status via the public DB read API.
41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":

```

```

94         is_valid, errors = self.validator.validate_tennis_points(annotations)
95     elif sport_str == "cricket":
96         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97     elif sport_str == "pickleball":
98         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99     elif sport_str == "tabletennis":
100         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102     if not is_valid:
103         print(f"Error: Annotations failed validation!")
104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0
127     if sport_str == "badminton":
128         count = self.db.insert_badminton_rallies(annotations)
129     elif sport_str == "tennis":
130         count = self.db.insert_tennis_points(annotations)
131     elif sport_str == "cricket":
132         count = self.db.insert_cricket_deliveries(annotations)
133     elif sport_str == "pickleball":
134         count = self.db.insert_pickleball_rallies(annotations)
135     elif sport_str == "tabletennis":
136         count = self.db.insert_tabletennis_rallies(annotations)
137
138     print(f"\nSuccessfully imported local video '{match_name}':")
139     print(f" - Video ID: {video_id}")
140     print(f" - Sport: {sport_str}")
141     print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142     print(f" - Registered {count} annotated segments.\n")
143
144     # Per-video observability report (next to the video). Never raises.
145     from src.reporting import emit_import_report
146     src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147     report_paths = emit_import_report(
148         video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149         errors=[], count=count, sport=sport_str, source_format=src_fmt,
150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151     )
152     if report_paths:
153         print(f" - Observability report: {report_paths.get('technical_md')}")
154         print(f" - Customer summary : {report_paths.get('customer_md')}\n")
155

```

```

156     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157         """Parses simple JSON or CSV annotations files."""
158         parsed_items = []
159         ext = os.path.splitext(filepath)[1].lower()
160
161         if ext == '.json':
162             with open(filepath, 'r', encoding='utf-8') as f:
163                 data = json.load(f)
164
165             if not isinstance(data, list):
166                 raise ValueError("JSON file must contain a list of objects.")
167
168             # Uses the same safe casts as the CSV path: a missing/blank required
169             # time raises a clear ValueError (caught by the caller) instead of a
170             # bare KeyError, and optional ints degrade to sensible defaults.
171             for idx, obj in enumerate(data):
172                 if sport == "badminton":
173                     parsed_items.append(BadmintonRally(
174                         video_id=video_id,
175                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                         start_time=_safe_float(obj.get("start_time")),
177                         end_time=_safe_float(obj.get("end_time")),
178                         score_before=obj.get("score_before"),
179                         score_after=obj.get("score_after"),
180                         shots_count=_safe_int(obj.get("shots_count"), None),
181                         ending_reason=obj.get("ending_reason"),
182                         last_shot_outcome=obj.get("last_shot_outcome")
183                     ))
184                 elif sport == "tennis":
185                     parsed_items.append(TennisPoint(
186                         video_id=video_id,
187                         point_number=_safe_int(obj.get("point_number"), idx + 1),
188                         start_time=_safe_float(obj.get("start_time")),
189                         end_time=_safe_float(obj.get("end_time")),
190                         set_score=obj.get("set_score"),
191                         game_score=obj.get("game_score"),
192                         server=obj.get("server"),
193                         ending_type=obj.get("ending_type")
194                     ))
195                 elif sport == "cricket":
196                     parsed_items.append(CricketDelivery(
197                         video_id=video_id,
198                         over=_safe_int(obj.get("over"), 0),
199                         ball=_safe_int(obj.get("ball"), 0),
200                         start_time=_safe_float(obj.get("start_time")),
201                         end_time=_safe_float(obj.get("end_time")),
202                         batsman=obj.get("batsman"),
203                         bowler=obj.get("bowler"),
204                         runs=_safe_int(obj.get("runs"), None),
205                         wicket=bool(obj.get("wicket", False)),
206                         extras=_safe_int(obj.get("extras"), 0)
207                     ))
208                 elif sport == "pickleball":
209                     parsed_items.append(PickleballRally(
210                         video_id=video_id,
211                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212                         start_time=_safe_float(obj.get("start_time")),
213                         end_time=_safe_float(obj.get("end_time")),
214                         score_before=obj.get("score_before"),
215                         score_after=obj.get("score_after"),
216                         shots_count=_safe_int(obj.get("shots_count"), None),
217                         ending_reason=obj.get("ending_reason"),
218                         last_shot_outcome=obj.get("last_shot_outcome")
219                     ))

```

```

elif sport == "tabletennis":
    parsed_items.append(TableTennisRally(
        video_id=video_id,
        rally_number=_safe_int(obj.get("rally_number"), idx + 1),
        start_time=_safe_float(obj.get("start_time")),
        end_time=_safe_float(obj.get("end_time")),
        score_before=obj.get("score_before"),
        score_after=obj.get("score_after"),
        shots_count=_safe_int(obj.get("shots_count"), None),
        ending_reason=obj.get("ending_reason"),
        last_shot_outcome=obj.get("last_shot_outcome")
    ))
elif ext == '.csv':
    with open(filepath, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        # Normalize headers
        reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []

    for idx, row in enumerate(reader):
        if sport == "badminton":
            parsed_items.append(BadmintonRally(
                video_id=video_id,
                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                start_time=_safe_float(row.get("start_time")),
                end_time=_safe_float(row.get("end_time")),
                score_before=row.get("score_before"),
                score_after=row.get("score_after"),
                shots_count=_safe_int(row.get("shots_count"), None),
                ending_reason=row.get("ending_reason"),
                last_shot_outcome=row.get("last_shot_outcome")
            ))
        elif sport == "tennis":
            parsed_items.append(TennisPoint(
                video_id=video_id,
                point_number=_safe_int(row.get("point_number"), idx + 1),
                start_time=_safe_float(row.get("start_time")),
                end_time=_safe_float(row.get("end_time")),
                set_score=row.get("set_score"),
                game_score=row.get("game_score"),
                server=row.get("server"),
                ending_type=row.get("ending_type")
            ))
        elif sport == "cricket":
            parsed_items.append(CricketDelivery(
                video_id=video_id,
                over=_safe_int(row.get("over"), 0),
                ball=_safe_int(row.get("ball"), 0),
                start_time=_safe_float(row.get("start_time")),
                end_time=_safe_float(row.get("end_time")),
                batsman=row.get("batsman"),
                bowler=row.get("bowler"),
                runs=_safe_int(row.get("runs"), None),
                wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
                extras=_safe_int(row.get("extras"), 0)
            ))
        elif sport == "pickleball":
            parsed_items.append(PickleballRally(
                video_id=video_id,
                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                start_time=_safe_float(row.get("start_time")),
                end_time=_safe_float(row.get("end_time")),
                score_before=row.get("score_before"),
                score_after=row.get("score_after"),

```



```

283             shots_count=_safe_int(row.get("shots_count"), None),
284             ending_reason=row.get("ending_reason"),
285             last_shot_outcome=row.get("last_shot_outcome")
286         ))
287     elif sport == "tabletennis":
288         parsed_items.append(TableTennisRally(
289             video_id=video_id,
290             rally_number=_safe_int(row.get("rally_number"), idx + 1),
291             start_time=_safe_float(row.get("start_time")),
292             end_time=_safe_float(row.get("end_time")),
293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"
315     for i in range(attempts):
316         result = download_best_video(
317             url, dest_path,
318             max_height=max_height, max_size_mb=max_size_mb,
319             cookies_from_browser=args.cookies_from_browser,
320             cookies_file=args.cookies,
321             player_client=args.player_client,
322         )
323         if result.success and (result.height is None or result.height >=
args.min_height):
324             return result, None
325
326         if not result.success:
327             reason = result.error or "unknown error"
328         else:
329             reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
330                      f"YouTube served a low format (rate-limited?)")
331         try:
332             os.remove(result.path)
333         except OSError:
334             pass
335
336         if i < attempts - 1:
337             backoff = args.retry_backoff * (i + 1)
338             print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
339             time.sleep(backoff)
340     return None, reason
341
342 def fetch(self, args):
343     """(Re)download videos at best available resolution, updating the DB in place.

```

```

344
345     This is the in-place upgrade path: rows, rally annotations, and curation
346     status are all preserved ? only the video file plus its `local_path`,
347     `resolution`, and `fps` are refreshed. By default it skips videos that
348     already have a local file at or above `--min-height`, so re-runs are cheap
349     and resumable; `--force` re-fetches everything.
350     """
351     storage = self.config.get("video_storage", {})
352     local_dir = storage.get("local_dir", "./data/videos")
353     max_download_size_mb = storage.get("max_download_size_mb")
354     max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355     os.makedirs(local_dir, exist_ok=True)
356
357     commercial_only = not args.include_noncommercial
358     if args.status == "all":
359         statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
360     else:
361         statuses = [CurationStatus(args.status)]
362
363     supported = self.config.get("supported_sports", ["badminton"])
364     sport_strs = [args.sport] if args.sport else supported
365
366     # Collect target videos.
367     videos = []
368     for sport_str in sport_strs:
369         try:
370             sport = SportType(sport_str.lower())
371         except ValueError:
372             print(f"Warning: skipping unknown sport '{sport_str}'.")
373             continue
374         for status in statuses:
375             videos.extend(self.db.get_videos_by_status(status, sport))
376     if args.video_id:
377         videos = [v for v in videos if v.video_id == args.video_id]
378
379     if not videos:
380         print("\nNo matching videos to fetch.\n")
381         return
382
383     results = [] # (video_id, action, detail)
384     for video in videos:
385         if commercial_only and not video.is_commercial:
386             results.append((video.video_id, "skip", "non-commercial"))
387             continue
388         if not video.video_url or "watch?v=example" in video.video_url:
389             # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390             # had no URL for this match) ? nothing to download.
391             results.append((video.video_id, "skip", "no real source URL"))
392             continue
393
394         dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396         # Skip if we already have a good-enough local file (unless --force).
397         if not args.force and os.path.exists(dest_path):
398             _, height, _ = probe_resolution_fps(dest_path)
399             if height is not None and height >= args.min_height:
400                 results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
401                 continue
402
403         print(f"Fetching {video.video_id} ({video.match_name})...")
404         result, fail_reason = self._fetch_one_with_retry(
405             video.video_url, dest_path, args, max_height, max_download_size_mb)
406         if fail_reason is not None:

```

```

407         results.append((video.video_id, "FAIL", fail_reason))
408         if args.sleep:
409             time.sleep(args.sleep)
410         continue
411
412     # Update only the file-derived fields; preserve everything else (UPSERT
413     # mutates the row in place, leaving rallies intact).
414     video.local_path = result.path
415     if result.resolution_label:
416         video.resolution = result.resolution_label
417     if result.fps:
418         video.fps = result.fps
419     self.db.insert_video(video)
420
421     detail = (f"{result.resolution_label or '?'}"
422             f"{'f' {result.fps:g}fps' if result.fps else ''},
423 {result.size_mb:.0f} MB")
424     results.append((video.video_id, "fetched", detail))
425     if args.sleep:
426         time.sleep(args.sleep)
427
428     print("\n=== Fetch summary ===")
429     print(tabulate([[vid, action, detail] for vid, action, detail in results],
430                   headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
431     fetched = sum(1 for _, a, _ in results if a == "fetched")
432     failed = [r for r in results if r[1] == "FAIL"]
433     print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
434 results if a=='have')}}    "
435           f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}}    Failed :
436 {len(failed)}}")
437     if failed:
438         print("Failures (left unchanged in DB):")
439         for vid, _, detail in failed:
440             print(f"  - {vid}: {detail}")
441     print()
442
443 def export_eval_set(self, args):
444     """Export curated annotations as indexer-ready eval/training sets.
445
446     Emits, per qualifying video, a `//.csv` in the
447     indexer's `start,end` ground-truth format (decimal seconds), plus a single
448     `/manifest.json` that pairs each CSV with its video file. The
449     manifest is the contract the downstream eval/training driver reads: the
450     indexer keys videos by *file MD5*, not by our `video_id`, so it must run
451     against the exact `local_path`/`video_url` recorded here.
452
453     By default only CURATED + commercially-safe videos are exported, mirroring
454     the license gate already enforced elsewhere in the toolchain.
455     """
456     out_dir = args.out_dir
457     commercial_only = not args.include_noncommercial
458     statuses = [CurationStatus.CURATED]
459     if args.include_staging:
460         statuses.append(CurationStatus.STAGING)
461
462     supported = self.config.get("supported_sports", ["badminton"])
463     sport_strs = [args.sport] if args.sport else supported
464
465     manifest = []
466     skipped_noncomm = 0
467     skipped_empty = 0
468
469     for sport_str in sport_strs:
470         try:
471             sport = SportType(sport_str.lower())

```

```

469         except ValueError:
470             print(f"Warning: skipping unknown sport '{sport_str}'.")
471             continue
472
473     videos = []
474     for status in statuses:
475         videos.extend(self.db.get_videos_by_status(status, sport))
476
477     for video in videos:
478         if commercial_only and not video.is_commercial:
479             skipped_noncomm += 1
480             continue
481
482         intervals = self.db.get_segment_intervals(video.video_id, sport)
483         if not intervals:
484             skipped_empty += 1
485             continue
486
487         sport_dir = os.path.join(out_dir, sport.value)
488         os.makedirs(sport_dir, exist_ok=True)
489         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490         with open(csv_path, "w", newline="", encoding="utf-8") as f:
491             writer = csv.writer(f)
492             writer.writerow(["start", "end"])
493             for start, end in intervals:
494                 writer.writerow([start, end])
495
496         manifest.append({
497             "video_id": video.video_id,
498             "sport": sport.value,
499             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500             "local_path": video.local_path,
501             "video_url": video.video_url,
502             "match_name": video.match_name,
503             "fps": video.fps,
504             "resolution": video.resolution,
505             "license_type": video.license_type.value,
506             "is_commercial": video.is_commercial,
507             "status": video.status.value,
508             "num_segments": len(intervals),
509         })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]]
528         for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")

```

```

532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest          : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540
541         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,
554                             video_path=args.video_path, step=args.step)
555         except (FileNotFoundError, ValueError) as e:
556             print(f"Error: {e}")
557             sys.exit(1)
558
559         write_canonical_csv(result.records, args.out)
560
561         # Observability report next to the canonical CSV. Never raises.
562         from src.reporting import emit_delivery_report
563         src_name = os.path.splitext(os.path.basename(args.input))[0]
564         dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
565         emit_delivery_report(
566             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
567             records=result.records, issues=result.issues, fps=result.fps,
568             video_path=args.video_path,
569             video_duration_s=dur, source_format="cvat", step=result.step,
570         )
571
572         report = render_issue_report(result, os.path.basename(args.input))
573         if args.issues:
574             with open(args.issues, "w", encoding="utf-8") as f:
575                 f.write(report)
576         print(report)
577         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
578         if result.n_blockers:
579             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
580                 f"this file until they are resolved (by the vendor or manual
review).")
581         else:
582             print("\n No blockers. Next:")
583             print(f"    python -m src.cli.curate import-local --sport <sport> "
584                   f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
585
586     def validate_delivery(self, args):
587         """Run the full validation suite on a rally delivery and emit a report.
588
589         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
590         CSV (--csv). Writes, into --out-dir:
591         <id>_report.md    human report (scoreboard + issues + rallies to review)

```

```

591         <id>_report.json  machine summary (for CI / a bulk-ingest loop)
592         <id>_review.csv   the annotatable manual-verification sheet
593     Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
594     """
595     import json
596     from src.parsers.cvat.rally_cvat import (
597         convert, load_canonical_csv, detect_issues,
598         build_summary, render_validation_md, write_review_csv,
599     )
600     vid = args.video_id or "delivery"
601     fps = None
602     step = None
603     stop_frame = None
604     if args.input:
605         try:
606             result = convert(args.input, fps=args.fps, video_path=args.video_path)
607         except (FileNotFoundError, ValueError) as e:
608             print(f"Error: {e}")
609             sys.exit(1)
610         records, issues, fps = result.records, result.issues, result.fps
611         step = result.step
612         stop_frame = result.meta.get("stop_frame")
613     elif args.csv:
614         try:
615             records = load_canonical_csv(args.csv)
616         except FileNotFoundError as e:
617             print(f"Error: {e}")
618             sys.exit(1)
619         if not records:
620             print(f"Error: no usable rally rows in {args.csv}")
621             sys.exit(1)
622         issues = detect_issues(records)
623     else:
624         print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
625         sys.exit(1)
626
627     os.makedirs(args.out_dir, exist_ok=True)
628     summary = build_summary(records, issues, source=vid, fps=fps)
629     md = render_validation_md(records, issues, source=vid)
630     json_path = os.path.join(args.out_dir, f"{vid}_report.json")
631     md_path = os.path.join(args.out_dir, f"{vid}_report.md")
632     review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
633     with open(json_path, "w", encoding="utf-8") as f:
634         json.dump(summary, f, indent=2)
635     with open(md_path, "w", encoding="utf-8") as f:
636         f.write(md)
637     write_review_csv(records, issues, review_path)
638
639     # Observability report (envelope JSON + technical md + customer summary),
640     # colocated with the existing artifacts. Never raises.
641     from src.reporting import emit_delivery_report
642     duration_s = (stop_frame / fps) if (stop_frame and fps) else None
643     report_paths = emit_delivery_report(
644         out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
645         video_path=args.video_path, video_duration_s=duration_s,
646         source_format=("cvat" if args.input else "csv"), step=step,
647     )
648
649     print(md)
650     print(f"\nwrote:\n {md_path}\n {json_path}\n {review_path}")
651     if report_paths:
652         print(f" {report_paths.get('technical_md')}\n
653 {report_paths.get('customer_md')}\n {report_paths.get('json')}")
654     if summary["n_blockers"]:
655         print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be

```

```

resolved.")
655         sys.exit(2)
656         print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
657
658     def curate(self):
659         """Interactive loop to review and curate staging video annotations."""
660         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
661         if not staging_videos:
662             print("\nNo staging video annotations to review.\n")
663             return
664
665         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
666         table_data = []
667         for idx, video in enumerate(staging_videos):
668             table_data.append([
669                 idx + 1,
670                 video.video_id,
671                 video.match_name,
672                 video.sport.value,
673                 video.license_type.value,
674                 "YES" if video.is_commercial else "NO"
675             ])
676
677         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
678 "License", "Commercial Safe?"], tablefmt="grid"))
679         print("\nEnter a number (1-{}) to review details, or 'q' to
680 quit:".format(len(staging_videos)))
681
682         choice = input("> ").strip()
683         if choice.lower() == 'q':
684             return
685
686         try:
687             idx = int(choice) - 1
688             if idx < 0 or idx >= len(staging_videos):
689                 print("Invalid choice.")
690                 return
691             self._curate_video_interactive(staging_videos[idx])
692         except ValueError:
693             print("Invalid input.")
694
695     def _curate_video_interactive(self, video: VideoMetadata):
696         """Displays details of a specific staging video and prompts for curation
697 action."""
698
699         # Load and count events
700         rallies = []
701         pts = []
702         delivs = []
703         if video.sport == SportType.BADMINTON:
704             rallies = self.db.get_badminton_rallies(video.video_id)
705         elif video.sport == SportType.PICKLEBALL:
706             rallies = self.db.get_pickleball_rallies(video.video_id)
707         elif video.sport == SportType.TABLE_TENNIS:
708             rallies = self.db.get_tabletennis_rallies(video.video_id)
709         elif video.sport == SportType.TENNIS:
710             pts = self.db.get_tennis_points(video.video_id)
711         elif video.sport == SportType.CRICKET:
712             delivs = self.db.get_cricket_deliveries(video.video_id)
713
714         while True:
715             print("\n===== REVIEWING VIDEO =====")
716             print(f"Match Name : {video.match_name}")
717             print(f"Video ID : {video.video_id}")
718             print(f"Sport : {video.sport.value}")

```

```

716         print(f"Video URL      : {video.video_url or 'N/A'}")
717         print(f"Local Path    : {video.local_path or 'N/A'}")
718         print(f"License       : {video.license_type.value} ({video.license_url or 'no
URL'})")
719         print(f"Commercial OK: {video.is_commercial}")
720
721         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
722             print(f"Total Rallies: {len(rallies)}")
723             if rallies:
724                 print("Sample rallies:")
725                 sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
726                 print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
727             elif video.sport == SportType.TENNIS:
728                 print(f"Total Points : {len(pts)}")
729                 if pts:
730                     print("Sample points:")
731                     sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732                     print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733             elif video.sport == SportType.CRICKET:
734                 print(f"Total Balls : {len(delivs)}")
735                 if delivs:
736                     print("Sample deliveries:")
737                     sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                     print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740             print("\nChoose Curation Action:")
741             print(" [a] Approve (promote to CURATED)")
742             print(" [r] Reject (delete from database)")
743             print(" [p] Play a rally/point segment in browser")
744             print(" [k] Keep in Staging (do nothing and go back)")
745
746             action = input("> ").strip().lower()
747             if action == 'a':
748                 self.db.update_status(video.video_id, CurationStatus.CURATED)
749                 print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750                 break
751             elif action == 'r':
752                 self.db.delete_video(video.video_id)
753                 print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754                 break
755             elif action == 'k':
756                 print("Curation state unchanged.\n")
757                 break
758             elif action == 'p':
759                 self._play_segment_interactive(video, rallies, pts, delivs)
760             else:
761                 print("Invalid action selected.")
762
763         def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764             import webbrowser
765
766             # Get start/end based on user input
767             start_time = None
768             end_time = None
769             label = "segment"
770
771             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,

```



```

SportType.TABLE_TENNIS):
772         if not rallies:
773             print("No rallies to play.")
774             return
775         idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
776         try:
777             rally_num = int(idx_str)
778             rally = next((r for r in rallies if r.rally_number == rally_num), None)
779             if rally:
780                 start_time = rally.start_time
781                 end_time = rally.end_time
782                 label = f"Rally {rally_num}"
783         except ValueError:
784             pass
785     elif video.sport == SportType.TENNIS:
786         if not pts:
787             print("No points to play.")
788             return
789         idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
790         try:
791             pt_num = int(idx_str)
792             pt = next((p for p in pts if p.point_number == pt_num), None)
793             if pt:
794                 start_time = pt.start_time
795                 end_time = pt.end_time
796                 label = f"Point {pt_num}"
797         except ValueError:
798             pass
799     elif video.sport == SportType.CRICKET:
800         if not delivs:
801             print("No deliveries to play.")
802             return
803         over_str = input("Enter Over number: ").strip()
804         ball_str = input("Enter Ball number: ").strip()
805         try:
806             o_num = int(over_str)
807             b_num = int(ball_str)
808             d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
809 None)
810             if d:
811                 start_time = d.start_time
812                 end_time = d.end_time
813                 label = f"Over {o_num} Ball {b_num}"
814         except ValueError:
815             pass
816         if start_time is None or end_time is None:
817             print("Segment not found or invalid input.")
818             return
819
820     print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
821
822     # Check video location
823     play_url = None
824     if video.local_path and os.path.exists(video.local_path):
825         abs_path = os.path.abspath(video.local_path)
826         # Create file:// URL with media fragment (browsers support #t=start,end
827 fragment)
828         play_url = f"file:/// {abs_path.replace(os.sep,
829 '/' )}#t={start_time},{end_time}"
830     elif video.video_url:
831         # YouTube URL: seek to start time
832         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
833             t_sec = int(start_time)
834             sep = "&" if "?" in video.video_url else "?"

```

```

833         play_url = f"{video.video_url}{sep}t={t_sec}s"
834     else:
835         play_url = video.video_url
836
837     if play_url:
838         print(f"Opening browser: {play_url}")
839         webbrowser.open(play_url)
840     else:
841         print("Error: No local video file or remote URL available to play.")
842
843
844     def main():
845         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
846         subparsers = parser.add_subparsers(dest="command")
847
848         # Status command
849         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
850
851         # Curate command
852         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
853
854         # Import-local command
855         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
856         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
857         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871             help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873             help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875             help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",
877             help="Re-download even if a good-enough local file already
exists")
878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879             help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",

```

```

metavar="BROWSER",
881                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                     "A Premium login gives reliable best-resolution
downloads. "
883                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884     fetch_parser.add_argument("--cookies", metavar="FILE",
885                               help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
886     fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                               help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                               "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889     fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                               help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
891     fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
892                               help="Retries per video when a download fails or returns a
low format (default: 2)")
893     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
894                               help="Base backoff between retries; grows linearly per
attempt (default: 20)")
895
896     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
897     cvat_parser = subparsers.add_parser(
898         "convert-cvat",
899         help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
900     cvat_parser.add_argument("--input", required=True,
901                              help="Path to the CVAT XML export (image- or track-mode)")
902     cvat_parser.add_argument("--out", required=True,
903                              help="Path to write the canonical per-rally CSV (import-
local input)")
904     cvat_parser.add_argument("--video-path",
905                              help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
906     cvat_parser.add_argument("--fps", type=float,
907                              help="Override fps instead of probing the video")
908     cvat_parser.add_argument("--step", type=int,
909                              help="Frame sampling step (default: read from CVAT meta,
else inferred)")
910     cvat_parser.add_argument("--issues",
911                              help="Path to also write the human-readable issues report")
912
913     # Validate-delivery command ? run the full check suite + emit a structured report
914     vd_parser = subparsers.add_parser(
915         "validate-delivery",
916         help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
917     vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
918     vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
919     vd_parser.add_argument("--video-path",
920                              help="Source video; fps via ffprobe (when using --input)")
921     vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
922     vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
923     vd_parser.add_argument("--out-dir", default="data/validation",
924                              help="Where to write the report + review sheet")

```

```

925
926     # Export command
927     export_parser = subparsers.add_parser(
928         "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
929     export_parser.add_argument("--out-dir", default="data/eval_export",
930                               help="Output directory for per-video CSVs and
manifest.json")
931     export_parser.add_argument("--sport",
932                               help="Restrict to a single sport (default: all
supported_sports)")
933     export_parser.add_argument("--include-staging", action="store_true",
934                               help="Also export videos still in staging (default:
curated only)")
935     export_parser.add_argument("--include-noncommercial", action="store_true",
936                               help="Include videos whose license is not commercially
safe (default: commercial only)")
937
938     args = parser.parse_args()
939     cli = SportsDataCLI()
940
941     if args.command == "status":
942         cli.status()
943     elif args.command == "curate":
944         cli.curate()
945     elif args.command == "import-local":
946         cli.import_local(args)
947     elif args.command == "convert-cvat":
948         cli.convert_cvat(args)
949     elif args.command == "validate-delivery":
950         cli.validate_delivery(args)
951     elif args.command == "export":
952         cli.export_eval_set(args)
953     elif args.command == "fetch":
954         cli.fetch(args)
955     else:
956         # Default: print help
957         parser.print_help()
958
959     if __name__ == "__main__":
960         main()
961

```

10. user (2026-06-10T05:33:40.596Z)

```

1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),

```

```

21     SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22 }
23
24
25 class SportsDatabase:
26     def __init__(self, db_path: str):
27         self.db_path = db_path
28         # Ensure target directory exists
29         os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30         self._init_db()
31
32     @contextmanager
33     def _get_connection(self):
34         """Yield a SQLite connection and always close it.
35
36         Used as ``with self._get_connection() as conn:``. A plain
37         ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38         transaction but does NOT close the connection, which leaks handles and
39         raises ResourceWarnings. This wrapper guarantees the connection is closed
40         on exit; writers still commit explicitly inside the block.
41         """
42         conn = sqlite3.connect(self.db_path)
43         conn.row_factory = sqlite3.Row
44         # SQLite disables foreign keys per-connection by default; enable so the
45         # ON DELETE CASCADE declarations on the child tables actually fire.
46         conn.execute("PRAGMA foreign_keys = ON")
47         try:
48             yield conn
49         finally:
50             conn.close()
51
52     def _init_db(self):
53         with self._get_connection() as conn:
54             cursor = conn.cursor()
55
56             # 1. Create common videos table
57             cursor.execute("""
58             CREATE TABLE IF NOT EXISTS videos (
59                 video_id TEXT PRIMARY KEY,
60                 sport TEXT NOT NULL,
61                 video_url TEXT,
62                 local_path TEXT,
63                 license_type TEXT NOT NULL,
64                 license_url TEXT,
65                 is_commercial INTEGER NOT NULL DEFAULT 0,
66                 status TEXT NOT NULL DEFAULT 'staging',
67                 match_name TEXT NOT NULL,
68                 fps REAL,
69                 resolution TEXT
70             )
71             """)
72
73             # 2. Create badminton_rallies table
74             cursor.execute("""
75             CREATE TABLE IF NOT EXISTS badminton_rallies (
76                 video_id TEXT NOT NULL,
77                 rally_number INTEGER NOT NULL,
78                 start_time REAL NOT NULL,
79                 end_time REAL NOT NULL,
80                 score_before TEXT,
81                 score_after TEXT,
82                 shots_count INTEGER,
83                 ending_reason TEXT,
84                 last_shot_outcome TEXT,
85                 PRIMARY KEY (video_id, rally_number),

```

```

86         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
87     )
88     """)
89
90 # 3. Create tennis_points table
91 cursor.execute("""
92 CREATE TABLE IF NOT EXISTS tennis_points (
93     video_id TEXT NOT NULL,
94     point_number INTEGER NOT NULL,
95     start_time REAL NOT NULL,
96     end_time REAL NOT NULL,
97     set_score TEXT,
98     game_score TEXT,
99     server TEXT,
100     ending_type TEXT,
101     PRIMARY KEY (video_id, point_number),
102     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
103 )
104 """)
105
106 # 4. Create cricket_deliveries table
107 cursor.execute("""
108 CREATE TABLE IF NOT EXISTS cricket_deliveries (
109     video_id TEXT NOT NULL,
110     over_num INTEGER NOT NULL,
111     ball_num INTEGER NOT NULL,
112     start_time REAL NOT NULL,
113     end_time REAL NOT NULL,
114     batsman TEXT,
115     bowler TEXT,
116     runs INTEGER,
117     wicket INTEGER DEFAULT 0,
118     extras INTEGER DEFAULT 0,
119     PRIMARY KEY (video_id, over_num, ball_num),
120     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
121 )
122 """)
123
124 # 5. Create pickleball_rallies table
125 cursor.execute("""
126 CREATE TABLE IF NOT EXISTS pickleball_rallies (
127     video_id TEXT NOT NULL,
128     rally_number INTEGER NOT NULL,
129     start_time REAL NOT NULL,
130     end_time REAL NOT NULL,
131     score_before TEXT,
132     score_after TEXT,
133     shots_count INTEGER,
134     ending_reason TEXT,
135     last_shot_outcome TEXT,
136     PRIMARY KEY (video_id, rally_number),
137     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
138 )
139 """)
140
141 # 6. Create tabletennis_rallies table
142 cursor.execute("""
143 CREATE TABLE IF NOT EXISTS tabletennis_rallies (
144     video_id TEXT NOT NULL,
145     rally_number INTEGER NOT NULL,
146     start_time REAL NOT NULL,
147     end_time REAL NOT NULL,
148     score_before TEXT,
149     score_after TEXT,
150     shots_count INTEGER,

```

```

151         ending_reason TEXT,
152         last_shot_outcome TEXT,
153         PRIMARY KEY (video_id, rally_number),
154         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
155     )
156     """
157
158     # Migration: add last_shot_outcome to rally tables that predate the
159     # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
160     # do it explicitly and idempotently (skip if the column is already there).
161     for tbl in ("badminton_rallies", "pickleball_rallies",
162 "tabletennis_rallies"):
163         existing = [r[1] for r in cursor.execute(f"PRAGMA
164 table_info({tbl}))"].fetchall()
165         if "last_shot_outcome" not in existing:
166             cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
167 TEXT")
168
169         conn.commit()
170
171 def insert_video(self, video: VideoMetadata) -> bool:
172     """Insert a video entry, or update it in place if it already exists.
173
174     Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
175     existing row, which (with foreign keys enabled) would cascade-delete the
176     video's child annotations. UPSERT mutates the row in place and leaves
177     children untouched.
178     """
179     with self._get_connection() as conn:
180         cursor = conn.cursor()
181         cursor.execute("""
182 INSERT INTO videos (
183     video_id, sport, video_url, local_path, license_type,
184     license_url, is_commercial, status, match_name, fps, resolution
185 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
186 ON CONFLICT(video_id) DO UPDATE SET
187     sport=excluded.sport,
188     video_url=excluded.video_url,
189     local_path=excluded.local_path,
190     license_type=excluded.license_type,
191     license_url=excluded.license_url,
192     is_commercial=excluded.is_commercial,
193     status=excluded.status,
194     match_name=excluded.match_name,
195     fps=excluded.fps,
196     resolution=excluded.resolution
197 """, (
198     video.video_id,
199     video.sport.value,
200     video.video_url,
201     video.local_path,
202     video.license_type.value,
203     video.license_url,
204     1 if video.is_commercial else 0,
205     video.status.value,
206     video.match_name,
207     video.fps,
208     video.resolution
209 ))
210         conn.commit()
211         return True
212
213 def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
214     if not rallies:
215         return 0

```

```

213         with self._get_connection() as conn:
214             cursor = conn.cursor()
215             # First clear existing rallies for this video_id to avoid key conflicts on
updates
216             video_id = rallies[0].video_id
217             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
218
219             for rally in rallies:
220                 cursor.execute("""
221                     INSERT INTO badminton_rallies (
222                         video_id, rally_number, start_time, end_time,
223                         score_before, score_after, shots_count, ending_reason,
last_shot_outcome
224                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
225                 """, (
226                     rally.video_id,
227                     rally.rally_number,
228                     rally.start_time,
229                     rally.end_time,
230                     rally.score_before,
231                     rally.score_after,
232                     rally.shots_count,
233                     rally.ending_reason,
234                     rally.last_shot_outcome
235                 ))
236             conn.commit()
237             return len(rallies)
238
239     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240         if not points:
241             return 0
242         with self._get_connection() as conn:
243             cursor = conn.cursor()
244             video_id = points[0].video_id
245             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
246
247             for pt in points:
248                 cursor.execute("""
249                     INSERT INTO tennis_points (
250                         video_id, point_number, start_time, end_time,
251                         set_score, game_score, server, ending_type
252                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
253                 """, (
254                     pt.video_id,
255                     pt.point_number,
256                     pt.start_time,
257                     pt.end_time,
258                     pt.set_score,
259                     pt.game_score,
260                     pt.server,
261                     pt.ending_type
262                 ))
263             conn.commit()
264             return len(points)
265
266     def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
267         if not deliveries:
268             return 0
269         with self._get_connection() as conn:
270             cursor = conn.cursor()
271             video_id = deliveries[0].video_id
272             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
273

```



```

274         for deliv in deliveries:
275             cursor.execute("""
276                 INSERT INTO cricket_deliveries (
277                     video_id, over_num, ball_num, start_time, end_time,
278                     batsman, bowler, runs, wicket, extras
279                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
280             """, (
281                 deliv.video_id,
282                 deliv.over,
283                 deliv.ball,
284                 deliv.start_time,
285                 deliv.end_time,
286                 deliv.batsman,
287                 deliv.bowler,
288                 deliv.runs,
289                 1 if deliv.wicket else 0,
290                 deliv.extras
291             ))
292         conn.commit()
293         return len(deliveries)
294
295     def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
296         if not rallies:
297             return 0
298         with self._get_connection() as conn:
299             cursor = conn.cursor()
300             video_id = rallies[0].video_id
301             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
302 (video_id,))
303
304         for rally in rallies:
305             cursor.execute("""
306                 INSERT INTO pickleball_rallies (
307                     video_id, rally_number, start_time, end_time,
308                     score_before, score_after, shots_count, ending_reason,
309                     last_shot_outcome
310                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
311             """, (
312                 rally.video_id,
313                 rally.rally_number,
314                 rally.start_time,
315                 rally.end_time,
316                 rally.score_before,
317                 rally.score_after,
318                 rally.shots_count,
319                 rally.ending_reason,
320                 rally.last_shot_outcome
321             ))
322             conn.commit()
323             return len(rallies)
324
325     def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
326         if not rallies:
327             return 0
328         with self._get_connection() as conn:
329             cursor = conn.cursor()
330             video_id = rallies[0].video_id
331             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
332 (video_id,))
333
334         for rally in rallies:
335             cursor.execute("""
336                 INSERT INTO tabletennis_rallies (
337                     video_id, rally_number, start_time, end_time,
338                     score_before, score_after, shots_count, ending_reason,

```

```

last_shot_outcome
336         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
337         """ , (
338             rally.video_id,
339             rally.rally_number,
340             rally.start_time,
341             rally.end_time,
342             rally.score_before,
343             rally.score_after,
344             rally.shots_count,
345             rally.ending_reason,
346             rally.last_shot_outcome
347         ))
348     conn.commit()
349     return len(rallies)
350
351     def get_video(self, video_id: str) -> Optional[VideoMetadata]:
352         with self._get_connection() as conn:
353             cursor = conn.cursor()
354             cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
355             row = cursor.fetchone()
356             if row:
357                 return VideoMetadata(
358                     video_id=row['video_id'],
359                     sport=SportType(row['sport']),
360                     video_url=row['video_url'],
361                     local_path=row['local_path'],
362                     license_type=LicenseType(row['license_type']),
363                     license_url=row['license_url'],
364                     is_commercial=bool(row['is_commercial']),
365                     status=CurationStatus(row['status']),
366                     match_name=row['match_name'],
367                     fps=row['fps'],
368                     resolution=row['resolution']
369                 )
370             return None
371
372     def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
373         query = "SELECT * FROM videos WHERE status = ?"
374         params = [status.value]
375         if sport:
376             query += " AND sport = ?"
377             params.append(sport.value)
378
379         videos = []
380         with self._get_connection() as conn:
381             cursor = conn.cursor()
382             cursor.execute(query, params)
383             for row in cursor.fetchall():
384                 videos.append(VideoMetadata(
385                     video_id=row['video_id'],
386                     sport=SportType(row['sport']),
387                     video_url=row['video_url'],
388                     local_path=row['local_path'],
389                     license_type=LicenseType(row['license_type']),
390                     license_url=row['license_url'],
391                     is_commercial=bool(row['is_commercial']),
392                     status=CurationStatus(row['status']),
393                     match_name=row['match_name'],
394                     fps=row['fps'],
395                     resolution=row['resolution']
396                 ))
397         return videos
398

```

```

399 def get_status_counts(self) -> List[tuple]:
400     """Return (sport, status, count) rows for the status summary.
401
402     Public read API so callers (e.g. the CLI `status` command) don't have to
403     reach into the private `_get_connection` context manager.
404     """
405     with self._get_connection() as conn:
406         cursor = conn.cursor()
407         cursor.execute(
408             "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
409             )
410         return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]
411
412 def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
413     rallies = []
414     with self._get_connection() as conn:
415         cursor = conn.cursor()
416         cursor.execute("""
417             SELECT * FROM badminton_rallies
418             WHERE video_id = ?
419             ORDER BY rally_number
420             """, (video_id,))
421         for row in cursor.fetchall():
422             rallies.append(BadmintonRally(
423                 video_id=row['video_id'],
424                 rally_number=row['rally_number'],
425                 start_time=row['start_time'],
426                 end_time=row['end_time'],
427                 score_before=row['score_before'],
428                 score_after=row['score_after'],
429                 shots_count=row['shots_count'],
430                 ending_reason=row['ending_reason'],
431                 last_shot_outcome=row['last_shot_outcome']
432             ))
433     return rallies
434
435 def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
436     points = []
437     with self._get_connection() as conn:
438         cursor = conn.cursor()
439         cursor.execute("""
440             SELECT * FROM tennis_points
441             WHERE video_id = ?
442             ORDER BY point_number
443             """, (video_id,))
444         for row in cursor.fetchall():
445             points.append(TennisPoint(
446                 video_id=row['video_id'],
447                 point_number=row['point_number'],
448                 start_time=row['start_time'],
449                 end_time=row['end_time'],
450                 set_score=row['set_score'],
451                 game_score=row['game_score'],
452                 server=row['server'],
453                 ending_type=row['ending_type']
454             ))
455     return points
456
457 def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
458     deliveries = []
459     with self._get_connection() as conn:
460         cursor = conn.cursor()
461         cursor.execute("""
462             SELECT * FROM cricket_deliveries

```

```

463         WHERE video_id = ?
464         ORDER BY over_num, ball_num
465         """, (video_id,))
466     for row in cursor.fetchall():
467         deliveries.append(CricketDelivery(
468             video_id=row['video_id'],
469             over=row['over_num'],
470             ball=row['ball_num'],
471             start_time=row['start_time'],
472             end_time=row['end_time'],
473             batsman=row['batsman'],
474             bowler=row['bowler'],
475             runs=row['runs'],
476             wicket=bool(row['wicket']),
477             extras=row['extras']
478         ))
479     return deliveries
480
481 def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
482     rallies = []
483     with self._get_connection() as conn:
484         cursor = conn.cursor()
485         cursor.execute("""
486         SELECT * FROM pickleball_rallies
487         WHERE video_id = ?
488         ORDER BY rally_number
489         """, (video_id,))
490     for row in cursor.fetchall():
491         rallies.append(PickleballRally(
492             video_id=row['video_id'],
493             rally_number=row['rally_number'],
494             start_time=row['start_time'],
495             end_time=row['end_time'],
496             score_before=row['score_before'],
497             score_after=row['score_after'],
498             shots_count=row['shots_count'],
499             ending_reason=row['ending_reason'],
500             last_shot_outcome=row['last_shot_outcome']
501         ))
502     return rallies
503
504 def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
505     rallies = []
506     with self._get_connection() as conn:
507         cursor = conn.cursor()
508         cursor.execute("""
509         SELECT * FROM tabletennis_rallies
510         WHERE video_id = ?
511         ORDER BY rally_number
512         """, (video_id,))
513     for row in cursor.fetchall():
514         rallies.append(TableTennisRally(
515             video_id=row['video_id'],
516             rally_number=row['rally_number'],
517             start_time=row['start_time'],
518             end_time=row['end_time'],
519             score_before=row['score_before'],
520             score_after=row['score_after'],
521             shots_count=row['shots_count'],
522             ending_reason=row['ending_reason'],
523             last_shot_outcome=row['last_shot_outcome']
524         ))
525     return rallies
526
527 def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:

```

```

528         """Return chronologically-ordered (start_time, end_time) tuples for a video,
529         regardless of sport. This is the sport-agnostic view used to export
530         indexer-ready evaluation/training labels.
531
532         Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533         yield an empty list rather than raising.
534         """
535         mapping = SEGMENT_TABLES.get(sport)
536         if mapping is None:
537             return []
538         table, order_by = mapping # both come from the trusted map above, never user
input
539         with self._get_connection() as conn:
540             cursor = conn.cursor()
541             cursor.execute(
542                 f"SELECT start_time, end_time FROM {table} "
543                 f"WHERE video_id = ? ORDER BY {order_by}",
544                 (video_id,),
545             )
546             return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547
548     def update_status(self, video_id: str, status: CurationStatus) -> bool:
549         with self._get_connection() as conn:
550             cursor = conn.cursor()
551             cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
(status.value, video_id))
552             conn.commit()
553             return cursor.rowcount > 0
554
555     def delete_video(self, video_id: str) -> bool:
556         """Deletes a video and cascading child entries."""
557         with self._get_connection() as conn:
558             cursor = conn.cursor()
559             # Execute cascade deletes manually just in case SQLite foreign key support
is not active
560             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
561             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
562             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
563             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
564             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
565             cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
566             conn.commit()
567             return cursor.rowcount > 0
568

```

11. user (2026-06-10T05:33:47.272Z)

```

1     """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3     Why this exists
4     -----
5     Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6     per-rally **CSV** that imports directly via ``curate import-local``. A vendor may
7     instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
8     boxes, with the rally fields (`rally_number`, `shots_count`,
9     `ending_reason` ...) carried as CVAT **object attributes** on each box. The box
10    geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11    metadata to.
12
13    This module is the ingestion-side adapter that absorbs that vendor format so the
14    modeling repo never has to. It does two things, both deliberately on *our* side

```

```

15     so we fix mechanical problems rather than renegotiating them:
16
17     1. **Recompute time from the frame range.** A rally's interval is taken from the
18         first/last *frame* a box for that rally appears on:
19         ``seconds = frame / fps`` at the video's true fps. We **ignore** any
20         ``start_time``/``end_time`` attributes the vendor wrote ? a CVAT job that
21         samples every ``step`` frames and assumes 30 fps yields systematically wrong
22         times (we have seen ``start_time == frame / (step * 30)`` , i.e. off by the
23         sampling step at the wrong fps). The frame index is reliable; the vendor's
24         seconds are not.
25
26     2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
27         zero-duration rallies, ending_reason values outside the agreed set, and
28         too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29         rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either **image mode**
32     (``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import os
39     import xml.etree.ElementTree as ET
40     from collections import Counter
41     from dataclasses import dataclass, field
42     from typing import Any, Dict, List, Optional, Tuple
43
44     from src.core.parsing import safe_int
45     from src.core.downloader import probe_resolution_fps
46
47     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48     # Kept here as a soft check only (a warning, never a hard failure) so this
49     # adapter does not depend on the annotation-spec wording.
50     DEFAULT_ALLOWED_REASONS = {
51         "winner",
52         "forced_error",
53         "unforced_error",
54         "service_fault",
55         "let",
56         "other",
57     }
58
59     # The observational outcome of the last shot ? orthogonal to ending_reason
60     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63     # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64     # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65     _ATTR_ALIASES = {
66         "rally_number": ("rally_number",),
67         "shots_count": ("shots_count",),
68         "ending_reason": ("ending_reason",),
69         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70         "score_before": ("score_before", "score_before (a-b)"),
71         "score_after": ("score_after", "score_after (a-b)"),
72         "notes": ("notes", "note"),
73     }
74
75     CANONICAL_COLUMNS = [
76         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
78         "notes",

```

```

78 ]
79
80
81 @dataclass
82 class RallyRecord:
83     """One rally recovered from the CVAT export, time re-derived from frames."""
84     rally_number: int
85     start_time: float          # seconds, = frame_min / fps
86     end_time: float           # seconds, = frame_max / fps (see padding note)
87     shots_count: Optional[int]
88     ending_reason: Optional[str]
89     last_shot_outcome: Optional[str]
90     score_before: Optional[str]
91     score_after: Optional[str]
92     notes: Optional[str]
93     frame_min: int
94     frame_max: int
95     n_boxes: int
96     vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97     vendor_end_time: Optional[str] = None
98
99     @property
100     def duration(self) -> float:
101         return self.end_time - self.start_time
102
103
104 @dataclass
105 class Issue:
106     """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107     severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108     code: str
109     message: str
110     rally_number: Optional[int] = None
111
112
113 @dataclass
114 class ConversionResult:
115     records: List[RallyRecord]
116     issues: List[Issue]
117     fps: float
118     step: int
119     meta: Dict[str, Any] = field(default_factory=dict)
120
121     @property
122     def n_blockers(self) -> int:
123         return sum(1 for i in self.issues if i.severity == "blocker")
124
125
126 def _mmss(seconds: float) -> str:
127     """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128     m = int(seconds // 60)
129     s = seconds - m * 60
130     return f"{m}:{s:06.3f}"
131
132
133 def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134     for alias in _ATTR_ALIASES[canonical]:
135         if alias in attrs and attrs[alias] != "":
136             return attrs[alias]
137     return None
138
139
140 def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141     """Parse a CVAT XML export into (meta, boxes).
142

```

```

143     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144     entry per visible box, from either image-mode or track-mode exports.
145     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146     """
147     if not os.path.exists(xml_path):
148         raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150     root = ET.parse(xml_path).getroot()
151     meta: Dict[str, Any] = {}
152
153     job = root.find("./job")
154     if job is not None:
155         stop = job.findtext("stop_frame")
156         if stop is not None and stop.strip().isdigit():
157             meta["stop_frame"] = int(stop.strip())
158         ff = job.findtext("frame_filter") or ""
159         if "step=" in ff:
160             try:
161                 meta["step"] = int(ff.split("step=")[1].split()[0])
162             except (ValueError, IndexError):
163                 pass
164
165     def _attrs(el: ET.Element) -> Dict[str, str]:
166         out: Dict[str, str] = {}
167         for a in el.findall("attribute"):
168             out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169         return out
170
171     boxes: List[Dict[str, Any]] = []
172
173     # Image mode: <image id=frame> ... <box>...</box>
174     for img in root.findall("image"):
175         try:
176             frame = int(img.get("id"))
177         except (TypeError, ValueError):
178             continue
179         for box in img.findall("box"):
180             boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182     # Track mode: <track> ... <box frame=N outside=0>...</box>
183     for track in root.findall("track"):
184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):
190                 continue
191             boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196     def _infer_step(frames: List[int]) -> int:
197         """Most common positive gap between consecutive distinct sampled frames."""
198         uniq = sorted(set(frames))
199         gaps = [b - a for a, b in zip(uniq, uniq[1:])] if b - a > 0
200         if not gaps:
201             return 1
202         return Counter(gaps).most_common(1)[0][0]
203
204
205     def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206         """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207         if fps is None or fps <= 0:

```



```

200         raise ValueError("fps must be a positive number to convert frames to seconds")
201
202     by_rally: Dict[int, List[Dict[str, Any]]] = {}
203     for b in boxes:
204         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
205         if rn is None:
206             continue
207         by_rally.setdefault(rn, []).append(b)
208
209     records: List[RallyRecord] = []
210     for rn in sorted(by_rally):
211         items = by_rally[rn]
212         frames = sorted(b["frame"] for b in items)
213         attrs = items[0]["attrs"]
214         fmin, fmax = frames[0], frames[-1]
215         records.append(RallyRecord(
216             rally_number=rn,
217             start_time=fmin / fps,
218             end_time=fmax / fps,
219             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
220             ending_reason=_attr_lookup(attrs, "ending_reason"),
221             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
222             score_before=_attr_lookup(attrs, "score_before"),
223             score_after=_attr_lookup(attrs, "score_after"),
224             notes=_attr_lookup(attrs, "notes"),
225             frame_min=fmin,
226             frame_max=fmax,
227             n_boxes=len(frames),
228             vendor_start_time=attrs.get("start_time"),
229             vendor_end_time=attrs.get("end_time"),
230         ))
231     return records
232
233 def detect_issues(records: List[RallyRecord],
234                  stop_frame: Optional[int] = None,
235                  allowed_reasons: Optional[set] = None,
236                  allowed_outcomes: Optional[set] = None,
237                  max_gap_s: float = 15.0) -> List[Issue]:
238     """Find the defects that block or degrade a clean ground-truth import.
239
240     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
241     slack of ~0.5s is acceptable for this pipeline because the downstream
242     highlight compiler pads every clip on output (badminton-highlight-indexer,
243     backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
244     absorbs the sampling uncertainty when highlights are trimmed.
245     """
246     allowed = allowed_reasons if allowed_reasons is not None else
247     DEFAULT_ALLOWED_REASONS
248     allowed_out = allowed_outcomes if allowed_outcomes is not None else
249     DEFAULT_ALLOWED_OUTCOMES
250     issues: List[Issue] = []
251
252     ordered = sorted(records, key=lambda r: r.rally_number)
253     for i, r in enumerate(ordered):
254         if r.end_time <= r.start_time:
255             issues.append(Issue(
256                 "blocker", "zero_duration",
257                 f"Rally {r.rally_number}: end <= start "
258                 f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
259                 r.rally_number))
260
261         if r.ending_reason is not None and r.ending_reason not in allowed:
262             issues.append(Issue(
263                 "medium", "ending_reason_vocab",

```

```

271         f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
272         f"not in the agreed set {sorted(allowed)}.",
273         r.rally_number))
274
275     if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
276         issues.append(Issue(
277             "low", "outcome_vocab",
278             f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
279             f"not in {sorted(allowed_out)}.",
280             r.rally_number))
281
282     # Inflated span: a long rally with implausibly few shots usually means a
283     # stray/duplicated box stretched the frame range past the real ending.
284     if r.shots_count and r.duration >= 5.0:
285         sps = r.shots_count / r.duration
286         if sps < 0.3:
287             issues.append(Issue(
288                 "medium", "shot_density",
289                 f"Rally {r.rally_number}: {r.shots_count} shots over "
290                 f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
291                 f"may be inflated by a stray box.",
292                 r.rally_number))
293
294     if stop_frame is not None and r.frame_max > stop_frame:
295         issues.append(Issue(
296             "high", "frame_out_of_range",
297             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
298             f"last frame {stop_frame}.",
299             r.rally_number))
300
301     if i > 0:
302         prev = ordered[i - 1]
303         if r.start_time < prev.end_time:
304             issues.append(Issue(
305                 "blocker", "overlap",
306                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
307                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
308                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
309                 r.rally_number))
310             elif r.start_time - prev.end_time > max_gap_s:
311                 # A long unlabeled stretch between two rallies is a cheap, label-only
312                 # second opinion for the worst error (a missed rally). It won't catch a
313                 # rally MERGED into its neighbour's span (shot_density/inflated catches
314                 # that), but it does catch an omitted rally that left a visible hole.
315                 gap = r.start_time - prev.end_time
316                 issues.append(Issue(
317                     "medium", "possible_missed_rally",
318                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
321                     gap.",
322                     r.rally_number))
323
324     return issues
325
326 def convert(xml_path: str,
327            fps: Optional[float] = None,
328            video_path: Optional[str] = None,
329            step: Optional[int] = None,
330            allowed_reasons: Optional[set] = None) -> ConversionResult:
331     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
332
333     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
334     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
335     that silent guess is exactly the bug this adapter exists to undo.

```

```

335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                             stop_frame=meta.get("stop_frame"),
352                             allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
354                             meta=meta)
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373
374 def render_issue_report(result: ConversionResult, source: str) -> str:
375     """A human-readable issue report (for review / sending back to the vendor)."""
376     lines = [
377         f"CVAT rally conversion report ? {source}",
378         f"  fps used      : {result.fps:.3f}",
379         f"  sampling step  : {result.step} frames (~{result.step / result.fps:.2f}s)",
380         f"  rallies parsed : {len(result.records)}",
381         f"  issues        : {len(result.issues)} ({result.n_blockers} blocker)",
382         "",
383     ]
384
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")
393             for i in group:
394                 lines.append(f"  - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398

```

```

399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total
416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):
432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437 [])]
438         for raw in reader:
439             row = {(k or "").strip().lower(): v for k, v in raw.items()}
440             rn = safe_int(row.get("rally_number"), None)
441             if rn is None:
442                 continue
443             start = _parse_secs(row.get("start_time"))
444             if start is None:
445                 start = _parse_secs(row.get("start_mmss"))
446             end = _parse_secs(row.get("end_time"))
447             if end is None:
448                 end = _parse_secs(row.get("end_mmss"))
449             if start is None or end is None:
450                 continue
451             records.append(RallyRecord(
452                 rally_number=rn, start_time=start, end_time=end,
453                 shots_count=safe_int(row.get("shots_count"), None),
454                 ending_reason=(row.get("ending_reason") or "").strip() or None,
455                 last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
456                 score_before=(row.get("score_before") or "").strip() or None,
457                 score_after=(row.get("score_after") or "").strip() or None,
458                 notes=(row.get("notes") or "").strip() or None,
459                 frame_min=-1, frame_max=-1, n_boxes=0,
460             ))
461     return records
462

```

```

463 def build_summary(records: List[RallyRecord], issues: List[Issue],
464                   source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
465     """Machine-readable validation summary (JSON / CI)."""
466     by_sev = {s: [i for i in issues if i.severity == s]
467              for s in ("blocker", "high", "medium", "low")}
468     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
469     return {
470         "source": source,
471         "fps": fps,
472         "n_rallies": len(records),
473         "n_issues": len(issues),
474         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
475         "n_blockers": len(by_sev["blocker"]),
476         "passed": len(by_sev["blocker"]) == 0,
477         "flagged_rallies": flagged,
478         "issues": [{"severity": i.severity, "code": i.code,
479                    "rally_number": i.rally_number, "message": i.message} for i in
480 issues],
481     }
482
483 def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
-> str:
484     """Human validation report: scoreboard + issues by severity + rallies to review."""
485     n = len(records)
486     by_sev = {s: [i for i in issues if i.severity == s]
487              for s in ("blocker", "high", "medium", "low")}
488     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
489     clean = n - len(flagged)
490     lines = [
491         f"# Delivery validation ? {source}",
492         "",
493         f"- rallies: **{n}**",
494         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
495         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
496         {len(by_sev['low'])})",
497         f"- structurally-clean rallies: **{clean}/{n}**",
498         + (f" ({100 * clean // n}%" if n else ""),
499         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
500         "",
501     ]
502     if issues:
503         for sev in ("blocker", "high", "medium", "low"):
504             grp = by_sev[sev]
505             if not grp:
506                 continue
507             lines.append(f"## {sev.upper()}")
508             lines += [f"- ({i.code}) {i.message}" for i in grp]
509             lines.append("")
510     else:
511         lines.append("No issues detected. ?\n")
512     if flagged:
513         lines.append(f"## Rallies to review: {flagged}")
514     return "\n".join(lines)
515
516 def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
517     """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""
518     import csv
519     flags: Dict[int, List[str]] = {}
520     for i in issues:
521         if i.rally_number is not None:
522             flags.setdefault(i.rally_number, []).append(i.code)
523     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)

```

```

524     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
525             "ending_reason", "last_shot_outcome", "auto_flags",
526             "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
527             "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
528     with open(out_path, "w", newline="", encoding="utf-8") as f:
529         w = csv.writer(f)
530         w.writerow(cols)
531         for r in sorted(records, key=lambda x: x.rally_number):
532             w.writerow([
533                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
534                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
535                 r.ending_reason or "", r.last_shot_outcome or "",
536                 ",".join(flags.get(r.rally_number, [])),
537                 "", "", "", "", "", "",
538             ])
539 
```

12. user (2026-06-10T05:33:52.750Z)

```

1     """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
2     ingests safely.
3
4     Two verified landmines in PARTIAL human annotation data (a video tagged only up to
5     some point, with mid-session corrections):
6
7     1. An unusable time aborts the WHOLE import (one bad row kills the entire video,
8         via the try/except around _parse_local_annotations in curate.py import-local):
9         a blank/missing time makes ``safe_float_required`` raise; a reversed
10        ``end <= start`` or a negative ``start`` makes the ``BadmintonRally`` pydantic
11        validator raise. -> we DROP such rows, mirroring the downstream indexer loader
12        which already skips ``not end > start``.
13
14    2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
15        the annotator missed) truncates via ``int(float())`` and COLLIDES on the
16        ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
17        silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
18        original id for traceability.
19
20    It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
21    pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
22    output is always the canonical lean schema::
23
24        rally_number,start_time,end_time,ending_reason,sport[,shots_count]
25
26    plus traceability columns (``original_rally_number``, ``notes``) that import-local
27    ignores. Standalone on purpose: it does NOT touch the DB schema or the
28    rally_number-ordered export. Run it, then feed the output to ``curate import-local``.
29
30    python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
31
32    """
33    from __future__ import annotations
34
35    import argparse
36    import csv
37    import os
38
39    from dataclasses import dataclass, field
40    from typing import Any, Dict, List, Optional, Tuple
41
42    # Canonical lean output columns + traceability (import-local reads the first four/
43    # five; original_rally_number + notes are ignored by the importer but kept for humans).
44    OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
45                  "shots_count", "original_rally_number", "notes"]
46
47    @dataclass
48    class NormalizeReport:

```

```

47     rows_in: int = 0
48     rows_out: int = 0
49     dropped: List[Dict[str, Any]] = field(default_factory=list) # {orig_id, reason,
start, end}
50     renumbered: bool = False
51     fractional_ids: List[str] = field(default_factory=list)
52     duplicate_ids: List[str] = field(default_factory=list)
53     mmss_converted: int = 0
54
55     def summary(self) -> str:
56         lines = [f"rows in: {self.rows_in} -> rows out: {self.rows_out}"]
57         if self.dropped:
58             lines.append(f"dropped {len(self.dropped)} bad row(s):")
59             for d in self.dropped:
60                 lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
61                             f"(start={d['start']!r}, end={d['end']!r})")
62         if self.renumbered:
63             extra = []
64             if self.fractional_ids:
65                 extra.append(f"fractional ids {self.fractional_ids}")
66             if self.duplicate_ids:
67                 extra.append(f"duplicate ids {self.duplicate_ids}")
68             lines.append("renumbered rallies 1..N by start_time"
69                         + (f" ({'; '.join(extra)})" if extra else ""))
70         if self.mmss_converted:
71             lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
72         if not self.dropped and not self.renumbered and not self.mmss_converted:
73             lines.append("clean input ? no changes beyond column normalization")
74         return "\n".join(lines)
75
76
77     def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
78         """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
79         if raw is None:
80             return None
81         s = str(raw).strip()
82         if not s:
83             return None
84         try:
85             if ":" in s:
86                 parts = [float(p) for p in s.split(":")]
87                 if len(parts) == 3:
88                     return parts[0] * 3600 + parts[1] * 60 + parts[2], True
89                 if len(parts) == 2:
90                     return parts[0] * 60 + parts[1], True
91                 return None
92             return float(s), False
93         except ValueError:
94             return None
95
96
97     def _primary_reason(raw: Any) -> Tuple[str, str]:
98         """Pipe-delimited 'forced_error|other' -> ('forced_error', 'other'). Blank -> ('',
'')."""
99         if raw is None:
100             return "", ""
101         s = str(raw).strip()
102         if not s:
103             return "", ""
104         parts = [p.strip() for p in s.split("|") if p.strip()]
105         if not parts:
106             return "", ""
107         return parts[0], ";" + ".join(parts[1:])

```

```

108
109
110 def _get(row: Dict[str, Any], *keys: str) -> Any:
111     """First non-empty value among the given (already-lowercased) keys."""
112     for k in keys:
113         v = row.get(k)
114         if v is not None and str(v).strip() != "":
115             return v
116     return None
117
118
119 def normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]],
NormalizeReport]:
120     """Pure core: lowercase-keyed annotator rows -> clean lean rows + report."""
121     rep = NormalizeReport(rows_in=len(rows))
122     kept: List[Dict[str, Any]] = []
123
124     for row in rows:
125         orig_id = _get(row, "rally_number", "rally")
126         s_parsed = _to_seconds(_get(row, "start_time", "start_mmss", "start"))
127         e_parsed = _to_seconds(_get(row, "end_time", "end_mmss", "end"))
128         start = s_parsed[0] if s_parsed else None
129         end = e_parsed[0] if e_parsed else None
130
131         if start is None:
132             rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
start_time",
133                               "start": _get(row, "start_time", "start_mmss"), "end":
_get(row, "end_time", "end_mmss")})
134             continue
135         if start < 0:
136             # BadmintonRally's pydantic validator rejects start<0 -> would abort the
137             # whole import; drop it here like any other unusable row.
138             rep.dropped.append({"orig_id": orig_id, "reason": "negative start_time",
139                               "start": start, "end": end})
140             continue
141         if end is None:
142             rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
end_time",
143                               "start": start, "end": _get(row, "end_time",
"end_mmss")})
144             continue
145         if end <= start:
146             rep.dropped.append({"orig_id": orig_id, "reason": "end_time <= start_time",
147                               "start": start, "end": end})
148             continue
149         if (s_parsed and s_parsed[1]):
150             rep.mmss_converted += 1
151         if (e_parsed and e_parsed[1]):
152             rep.mmss_converted += 1
153
154         reason, reason_note = _primary_reason(_get(row, "ending_reason"))
155         note_in = _get(row, "notes", "note", "reviewer_comment")
156         notes = "; ".join([x for x in [reason_note, (str(note_in).strip() if note_in
else "")] if x])
157         kept.append({
158             "orig_id": orig_id, "start": float(start), "end": float(end),
159             "ending_reason": reason, "sport": _get(row, "sport"),
160             "shots_count": _get(row, "shots_count", "shots"), "notes": notes,
161         })
162
163     kept.sort(key=lambda r: r["start"])
164
165     # Renumber 1..N by start order; detect whether ids were fractional / duplicate /
166     # not already a clean 1..N sequence (any of which means import-local would collide).

```



```

167 orig_seq = [str(r["orig_id"]) if r["orig_id"] is not None else "" for r in kept]
168 new_seq = [str(i) for i in range(1, len(kept) + 1)]
169 rep.renumbered = orig_seq != new_seq
170 seen: Dict[str, int] = {}
171 for oid in orig_seq:
172     seen[oid] = seen.get(oid, 0) + 1
173 rep.duplicate_ids = sorted({oid for oid, c in seen.items() if c > 1 and oid})
174 rep.fractional_ids = sorted({oid for oid in orig_seq if "." in oid})
175
176 out: List[Dict[str, Any]] = []
177 for i, r in enumerate(kept, start=1):
178     out.append({
179         "rally_number": i,
180         "start_time": f"{r['start']:.3f}",
181         "end_time": f"{r['end']:.3f}",
182         "ending_reason": r["ending_reason"] or "",
183         "sport": r["sport"] or "",
184         "shots_count": "" if r["shots_count"] is None else
str(r["shots_count"]).strip(),
185         "original_rally_number": "" if r["orig_id"] is None else str(r["orig_id"]),
186         "notes": r["notes"],
187     })
188 rep.rows_out = len(out)
189 return out, rep
190
191
192 def normalize_file(in_path: str, out_path: str) -> NormalizeReport:
193     with open(in_path, "r", newline="", encoding="utf-8-sig") as f:
194         reader = csv.DictReader(f)
195         reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or
[[]])]
196         rows = [{(k or "").lower().strip(): v for k, v in r.items()} for r in reader]
197     out, rep = normalize_rows(rows)
198     os.makedirs(os.path.dirname(os.path.abspath(out_path)) or ".", exist_ok=True)
199     with open(out_path, "w", newline="", encoding="utf-8") as f:
200         w = csv.DictWriter(f, fieldnames=OUT_FIELDS)
201         w.writeheader()
202         w.writerows(out)
203     return rep
204
205
206 def main() -> None:
207     ap = argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe
import-local.")
208     ap.add_argument("--in", dest="in_path", required=True, help="rally-annotator
.rallies.csv")
209     ap.add_argument("--out", dest="out_path", required=True, help="clean lean CSV to
write")
210     args = ap.parse_args()
211     rep = normalize_file(args.in_path, args.out_path)
212     print("== normalize-annotations ==")
213     print(rep.summary())
214     print(f"wrote -> {args.out_path}")
215
216
217 if __name__ == "__main__":
218     main()
219

```

13. user (2026-06-10T05:33:57.807Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.

```

```

5      """
6      from typing import Any, Optional
7
8
9      def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:
35             return 0.0
36         s = str(time_str).strip()
37         if ":" in s:
38             parts = s.split(":")
39             try:
40                 if len(parts) == 3:
41                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                 if len(parts) == 2:
43                     return float(parts[0]) * 60 + float(parts[1])
44             except ValueError:
45                 return 0.0
46         try:
47             return float(s)
48         except ValueError:
49             return 0.0
50

```

14. user (2026-06-10T05:33:58.213Z)

```

1      from typing import List, Tuple, Union
2      from src.core.config import load_config
3      from src.core.schemas.base import VideoMetadata, LicenseType
4      from src.core.schemas.badminton import BadmintonRally
5      from src.core.schemas.tennis import TennisPoint
6      from src.core.schemas.cricket import CricketDelivery
7      from src.core.schemas.pickleball import PickleballRally
8      from src.core.schemas.tabletennis import TableTennisRally
9
10     class DatasetValidator:
11         def __init__(self, config_path: str = "config/config.yaml"):
12             self.config_path = config_path
13             self.commercial_licenses = self._load_licenses_whitelist()

```

```

14
15     def _load_licenses_whitelist(self) -> List[str]:
16         # load_config returns {} on a missing/unreadable file, so the default list
17         # below is the single fallback (keep in sync with config.yaml).
18         config = load_config(self.config_path)
19         return config.get(
20             'commercial_licenses',
21             ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
22         )
23
24     def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
25         """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
26         val = license_type.value if isinstance(license_type, LicenseType) else
license_type
27         return val in self.commercial_licenses
28
29     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30         """
31         Validates badminton rallies:
32         - Sorted chronologically.
33         - Start time < End time (handled by Pydantic, but double checked).
34         - No temporal overlap between sequential rallies.
35         """
36         errors = []
37         if not rallies:
38             return True, errors
39
40         # Sort by rally number to check progression
41         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42
43         for i in range(len(sorted_rallies)):
44             r = sorted_rallies[i]
45             if r.start_time >= r.end_time:
46                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48             if i > 0:
49                 prev = sorted_rallies[i - 1]
50                 if r.start_time < prev.end_time:
51                     errors.append(
52                         f"Rally overlap detected between Rally {prev.rally_number} "
53                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                         f"(starts at {r.start_time}s)"
55                     )
56
57         return len(errors) == 0, errors
58
59     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70             if i > 0:
71                 prev = sorted_pts[i - 1]

```

```

72         if pt.start_time < prev.end_time:
73             errors.append(
74                 f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                 f"and Point {pt.point_number} (starts {pt.start_time}s)"
76             )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83
84         # Sort by over, then ball within over
85         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86         for i in range(len(sorted_delivs)):
87             d = sorted_delivs[i]
88             if d.start_time >= d.end_time:
89                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")
90
91             if i > 0:
92                 prev = sorted_delivs[i - 1]
93                 if d.start_time < prev.end_time:
94                     errors.append(
95                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
96                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
97                     )
98         return len(errors) == 0, errors
99
100    def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
101        """Validates pickleball rallies: positive durations and no temporal overlap
102        between chronologically-ordered rallies (same contract as badminton)."""
103        errors = []
104        if not rallies:
105            return True, errors
106
107        sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
108        for i in range(len(sorted_rallies)):
109            r = sorted_rallies[i]
110            if r.start_time >= r.end_time:
111                errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
112
113            if i > 0:
114                prev = sorted_rallies[i - 1]
115                if r.start_time < prev.end_time:
116                    errors.append(
117                        f"Rally overlap detected between Rally {prev.rally_number} "
118                        f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119                        f"(starts at {r.start_time}s)"
120                    )
121        return len(errors) == 0, errors
122
123    def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
124        """Validates table-tennis rallies: positive durations and no temporal overlap
125        between chronologically-ordered rallies (same contract as badminton)."""
126        errors = []
127        if not rallies:
128            return True, errors
129

```

```

130         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
131         for i in range(len(sorted_rallies)):
132             r = sorted_rallies[i]
133             if r.start_time >= r.end_time:
134                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
135
136             if i > 0:
137                 prev = sorted_rallies[i - 1]
138                 if r.start_time < prev.end_time:
139                     errors.append(
140                         f"Rally overlap detected between Rally {prev.rally_number} "
141                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
142                         f"(starts at {r.start_time}s)"
143                     )
144         return len(errors) == 0, errors
145

```

15. user (2026-06-10T05:34:03.627Z)

```

1     from pydantic import BaseModel, Field
2     from typing import Optional
3     from enum import Enum
4
5     class SportType(str, Enum):
6         BADMINTON = "badminton"
7         TENNIS = "tennis"
8         TABLE_TENNIS = "tabletennis"
9         PICKLEBALL = "pickleball"
10        CRICKET = "cricket"
11
12    class LicenseType(str, Enum):
13        MIT = "MIT"
14        APACHE_2 = "Apache-2.0"
15        BSD = "BSD"
16        CC0 = "CC0"
17        CC_BY = "CC-BY"
18        CC_BY_NC = "CC-BY-NC"
19        CC_BY_NC_SA = "CC-BY-NC-SA"
20        PUBLIC_DOMAIN = "Public-Domain"
21        PROPRIETARY = "Proprietary"
22        UNKNOWN = "Unknown"
23
24    class CurationStatus(str, Enum):
25        STAGING = "staging"
26        CURATED = "curated"
27        REJECTED = "rejected"
28
29    class VideoMetadata(BaseModel):
30        video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31        sport: SportType
32        video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33        local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34        license_type: LicenseType
35        license_url: Optional[str] = None
36        is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37        status: CurationStatus = CurationStatus.STAGING
38        match_name: str = Field(..., description="Name of the match/event")
39        fps: Optional[float] = Field(None, description="Frames per second")
40        resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")

```

16. user (2026-06-10T05:34:04.585Z)

```

1      from pydantic import BaseModel, Field, model_validator
2      from typing import Optional
3
4      class BadmintonRally(BaseModel):
5          video_id: str
6          rally_number: int = Field(..., description="Chronological rally index in the match")
7          start_time: float = Field(..., description="Start of rally in seconds")
8          end_time: float = Field(..., description="End of rally in seconds")
9          score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10         score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11         shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12         ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13         last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15         @model_validator(mode='after')
16         def validate_timestamps(self) -> 'BadmintonRally':
17             if self.start_time < 0:
18                 raise ValueError("start_time must be non-negative")
19             if self.end_time <= self.start_time:
20                 raise ValueError("end_time must be strictly greater than start_time")
21             return self
22

```

17. user (2026-06-10T05:34:04.664Z)

```

1      """Central configuration loader.
2
3      A single place to read ``config/config.yaml`` so modules don't each re-open the
4      file with slightly different fallbacks. Returns ``{}`` when the file is missing or
5      unreadable (callers apply their own defaults), matching the prior per-module
6      behaviour but without the duplicated boilerplate and broad excepts.
7      """
8      import os
9      from typing import Any, Dict
10
11      import yaml
12
13      DEFAULT_CONFIG_PATH = "config/config.yaml"
14      DEFAULT_DB_PATH = "data/sports_metadata.db"
15
16
17      def load_config(config_path: str = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
18          """Load a YAML config into a dict; return {} if missing/unreadable/empty."""
19          if not os.path.exists(config_path):
20              return {}
21          try:
22              with open(config_path, "r", encoding="utf-8") as f:
23                  return yaml.safe_load(f) or {}
24          except (OSError, yaml.YAMLError):
25              return {}
26
27
28      def get_database_path(config: Dict[str, Any]) -> str:
29          """Resolve the SQLite database path from a loaded config, with a default."""
30          return config.get("database_path", DEFAULT_DB_PATH)
31

```

18. user (2026-06-10T05:34:10.916Z)

```
1      """Adapt collector data into an obsreport envelope.
2
3      Reuses the existing rally_cvat report primitives (`Issue`, records, severities)
4      rather than re-deriving anything ? this module only *maps* them onto the shared
5      report shape. Imported only after `src.reporting` confirms obsreport is present,
6      so it may import obsreport freely.
7      """
8      from __future__ import annotations
9
10     from pathlib import Path
11     from typing import Any, List, Optional
12
13     import obsreport
14
15     _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
16
17     # Delivery issue severity -> report flag severity.
18     _SEVERITY = {"blocker": "error", "high": "warn", "medium": "warn", "low": "info"}
19     # Every issue code is documented in this FAQ table.
20     _ISSUE_FAQ = "docs/INGESTION_PIPELINE.md#what-does-validation-check"
21
22
23     def load_spec() -> "obsreport.ReportSpec":
24         return obsreport.load_spec(_SPEC_PATH)
25
26
27     # Heuristic checks that flag a region to REVIEW ? not hard failures. We frame these
28     # gently for the vendor so a "possible" finding isn't read as a certainty/blocker.
29     _REVIEW_ONLY = {"possible_missed_rally", "shot_density"}
30
31
32     def issue_to_flag(issue: Any, record: Any = None, fps: Any = None) -> "obsreport.Flag":
33         """Map a rally_cvat.Issue onto a report Flag. Actionable issues (blocker/high/
34         medium) also carry a customer_message so the delivery summary shows the vendor
35         exactly what to fix, in plain language. When the rally's source record + fps are
36         known, frame-level evidence is attached so a CVAT (frame-based) reviewer can act
37         without converting seconds to frames by hand."""
38         sev = _SEVERITY.get(issue.severity, "info")
39         customer = issue.message if sev in ("error", "warn") else None
40         if customer and issue.code in _REVIEW_ONLY:
41             customer += (" (Review only ? a heuristic, not a blocker: check the video and "
42                         "act only if confirmed.)")
43         evidence: dict = {}
44         if issue.rally_number is not None:
45             evidence["rally_number"] = issue.rally_number
46         if record is not None and getattr(record, "frame_min", -1) is not None and
47         getattr(record, "frame_min", -1) >= 0:
48             evidence["rally_frames"] = f"{record.frame_min}-{record.frame_max}"
49         if fps:
50             evidence["fps"] = fps
51         return obsreport.Flag(
52             code=issue.code,
53             severity=sev,
54             message=issue.message,
55             stage="validation",
56             faq_ref=_ISSUE_FAQ,
57             customer_message=customer,
58             evidence=evidence,
59         )
60
61     def _recorder(source: str, video_path: Optional[str], spec) -> "obsreport.RunRecorder":
62         return obsreport.RunRecorder(
```

```

63         "sports-data-collector",
64         video_id=source,
65         video_id_kind="human_id",
66         video_path=video_path,
67         spec=spec if spec is not None else load_spec(),
68     )
69
70
71 def build_delivery_recorder(
72     *,
73     records: List[Any],
74     issues: List[Any],
75     fps: Optional[float],
76     source: str,
77     video_path: Optional[str] = None,
78     video_duration_s: Optional[float] = None,
79     source_format: str = "cvat",
80     step: Optional[int] = None,
81     spec=None,
82 ) -> "obsreport.RunRecorder":
83     """Envelope for `validate-delivery` / `convert-cvat`: parse + validation stages,
84     one flag per delivery issue, a rally summary."""
85     rec = _recorder(source, video_path, spec)
86     if fps:
87         rec.set_video_meta(fps=fps, fps_source="declared")
88     if video_duration_s:
89         rec.set_video_meta(duration_s=round(video_duration_s, 1))
90
91     with rec.stage("parse") as st:
92         st.add_metric("rallies_parsed", len(records))
93         st.details.update({"source_format": source_format, "fps": fps, "sampling_step":
step})
94
95     blockers = [i for i in issues if i.severity == "blocker"]
96     with rec.stage("validation") as st:
97         for sev in ("blocker", "high", "medium", "low"):
98             st.add_metric(f"issues_{sev}", sum(1 for i in issues if i.severity == sev))
99         st.add_metric("n_blockers", len(blockers))
100         st.details["gate"] = "BLOCKED" if blockers else "PASS"
101         st.status = "error" if blockers else ("warn" if issues else "ok")
102
103     records_by_rally = {getattr(r, "rally_number", None): r for r in records}
104     for issue in issues:
105         record = records_by_rally.get(issue.rally_number)
106         rec.envelope.flags.append(issue_to_flag(issue, record=record, fps=fps))
107
108     _rally_summary(rec, records, video_duration_s)
109     rec.finalize()
110     return rec
111
112
113 def build_import_recorder(
114     *,
115     video_meta_obj: Any,
116     annotations: List[Any],
117     is_valid: bool,
118     errors: List[str],
119     count: int,
120     sport: str,
121     source_format: str,
122     spec=None,
123 ) -> "obsreport.RunRecorder":
124     """Envelope for `import-local`: parse + validation + registration stages."""
125     rec = _recorder(video_meta_obj.video_id, video_meta_obj.local_path, spec)
126     fps = getattr(video_meta_obj, "fps", None)

```



```

127     rec.set_video_meta(
128         fps=fps,
129         fps_source="declared" if fps else "unknown",
130         resolution=getattr(video_meta_obj, "resolution", None),
131     )
132
133     with rec.stage("parse") as st:
134         st.add_metric("records_parsed", len(annotations))
135         st.details["source_format"] = source_format
136
137     with rec.stage("validation") as st:
138         st.status = "ok" if is_valid else "error"
139         st.add_metric("valid", is_valid)
140         for e in errors or []:
141             st.messages.append(str(e))
142
143     with rec.stage("registration") as st:
144         st.add_metric("segments_registered", count)
145         st.details.update({
146             "sport": sport,
147             "license": getattr(getattr(video_meta_obj, "license_type", None), "value",
148                               str(getattr(video_meta_obj, "license_type", ""))),
149             "is_commercial": bool(getattr(video_meta_obj, "is_commercial", False)),
150             "status": getattr(getattr(video_meta_obj, "status", None), "value",
151                               str(getattr(video_meta_obj, "status", ""))),
152         })
153
154     _rally_summary(rec, annotations, None)
155     rec.finalize()
156     return rec
157
158
159     def _rally_summary(rec: "obsreport.RunRecorder", records: List[Any], video_duration_s:
Optional[float]) -> None:
160         total = 0.0
161         for r in records:
162             try:
163                 total += float(r.end_time) - float(r.start_time)
164             except (AttributeError, TypeError, ValueError):
165                 pass
166         coverage = round(100.0 * total / video_duration_s, 1) if video_duration_s else None
167         notes = [] if records else ["No rallies were found in this delivery."]
168         rec.set_highlights(segment_count=len(records), total_highlight_s=round(total, 1),
169                           coverage_pct=coverage, notes=notes)
170

```

19. user (2026-06-10T05:34:11.517Z)

```

1     """Per-video observability reports for the collector.
2
3     `obsreport` is a SOFT dependency: absent -> these emitters no-op and the CLI runs
4     unchanged. Every emitter is safe to call from normal control flow and NEVER raises
5     (emitting a report must not break ingestion). Returns the written paths, or None.
6     """
7     from __future__ import annotations
8
9     import os
10    from typing import Any, Dict, List, Optional
11
12    try: # soft dependency
13        import obsreport # noqa: F401
14        from . import adapter
15
16        OBSREPORT_AVAILABLE = True
17    except Exception: # pragma: no cover - only when obsreport is absent

```

```

18     OBSREPORT_AVAILABLE = False
19
20
21 def emit_delivery_report(
22     *,
23     out_dir: str,
24     source: str,
25     records: List[Any],
26     issues: List[Any],
27     fps: Optional[float],
28     video_path: Optional[str] = None,
29     video_duration_s: Optional[float] = None,
30     source_format: str = "cvat",
31     step: Optional[int] = None,
32 ) -> Optional[Dict[str, str]]:
33     """Write the delivery observability report into out_dir (alongside the existing
34     <id>_report.* artifacts). Used by validate-delivery / convert-cvat. Never raises."""
35     if not OBSREPORT_AVAILABLE:
36         return None
37     try:
38         rec = adapter.build_delivery_recorder(
39             records=records, issues=issues, fps=fps, source=source,
40             video_path=video_path,
41             video_duration_s=video_duration_s, source_format=source_format, step=step,
42         )
43         res = rec.write(output_dir=out_dir, stem=source)
44         return res.paths or None
45     except Exception: # pragma: no cover - reporting must never break the command
46         return None
47
48 def emit_import_report(
49     *,
50     video_meta_obj: Any,
51     annotations: List[Any],
52     is_valid: bool,
53     errors: List[str],
54     count: int,
55     sport: str,
56     source_format: str,
57     fallback_dir: Optional[str] = None,
58 ) -> Optional[Dict[str, str]]:
59     """Write the import observability report next to the imported video. If the video
60     file doesn't exist on disk (registration-only imports), fall back to
61     ``fallback_dir`` (e.g. the annotations directory) rather than the cwd. Never
62     raises."""
63     if not OBSREPORT_AVAILABLE:
64         return None
65     try:
66         rec = adapter.build_import_recorder(
67             video_meta_obj=video_meta_obj, annotations=annotations, is_valid=is_valid,
68             errors=errors, count=count, sport=sport, source_format=source_format,
69         )
70         video_path = getattr(video_meta_obj, "local_path", None)
71         if video_path and os.path.exists(video_path):
72             res = rec.write() # next to the video
73         else:
74             res = rec.write(output_dir=fallback_dir or ".",
75                             stem=getattr(video_meta_obj, "video_id", None))
76         return res.paths or None
77     except Exception: # pragma: no cover
78         return None

```

```

1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument("--license", default="MIT", help="Dataset license type")
13        parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14        parser.add_argument("--match-name", help="Name of the match")
15        parser.add_argument("--video-id", help="Video unique identifier")
16
17        args = parser.parse_args()
18
19        # Load db path from the central config (falls back to the default).
20        # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
21        # sports are populated via `curate import-local` or vendor golden sets.
22        db = SportsDatabase(get_database_path(load_config()))
23
24        if args.sport == "badminton" and args.parser == "coachai":
25            match_parser = CoachAIBadmintonParser()
26            try:
27                video_meta, rallies = match_parser.parse(
28                    args.file_path,
29                    video_id=args.video_id,
30                    video_url=args.video_url,
31                    license=args.license,
32                    match_name=args.match_name
33                )
34
35                # Insert into database (by default parsed entries are marked as staging)
36                db.insert_video(video_meta)
37                count = db.insert_badminton_rallies(rallies)
38
39                print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40                print(f" - Video ID: {video_meta.video_id}")
41                print(f" - Total Rallies: {count}")
42                print(f" - License: {video_meta.license_type.value}")
43                print(f" - Commercial safe: {video_meta.is_commercial}")
44            except Exception as e:
45                print(f"Error parsing file: {e}")
46                sys.exit(1)
47
48        if __name__ == "__main__":
49            main()
50

```

21. user (2026-06-10T05:34:21.072Z)

```

1     import csv
2     import os
3     from typing import Tuple, List, Dict, Any, Optional
4     from src.parsers.base_parser import BaseParser
5     from src.core.parsing import safe_int as _safe_int, parse_time_string
6     from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
7     from src.core.schemas.badminton import BadmintonRally

```

```

8     from src.core.validator import DatasetValidator
9
10
11     class CoachAIBadmintonParser(BaseParser):
12         def __init__(
13             self,
14             start_buffer: float = 1.0,
15             end_buffer: float = 2.0,
16             validator: Optional[DatasetValidator] = None,
17         ):
18             """
19             Args:
20                 start_buffer: Seconds to subtract from first stroke (serve) time.
21                 end_buffer: Seconds to add to last stroke time.
22                 validator: Used to decide commercial-license safety from the config
23                     whitelist. Defaults to a DatasetValidator with the default config so
24                     license classification stays consistent across the codebase.
25             """
26             self.start_buffer = start_buffer
27             self.end_buffer = end_buffer
28             self.validator = validator or DatasetValidator()
29
30         def parse(self, filepath: str, **kwargs) -> Tuple[VideoMetadata,
List[BadmintonRally]]:
31             """
32             Parses a CoachAI/ShuttleSet CSV format.
33
34             Expected CSV Columns (any subset or mapped names):
35                 - set: Set number (1, 2, 3)
36                 - rally: Rally number
37                 - ball_round: Shot number within the rally (1, 2, 3...)
38                 - time: Hitting time of the shot (in seconds)
39                 - type: Stroke type (e.g., smash, lob, net...)
40                 - score_a / score_b: Current scores
41             """
42             if not os.path.exists(filepath):
43                 raise FileNotFoundError(f"File not found: {filepath}")
44
45             # Extract video details from kwargs or name
46             match_name = kwargs.get("match_name") or
os.path.basename(filepath).replace(".csv", "")
47             video_id = kwargs.get("video_id") or match_name.lower().replace(" ", "_")
48             video_url = kwargs.get("video_url") or "https://youtube.com/watch?v=example"
49             license_str = kwargs.get("license") or "MIT"
50
51             # Check if license matches commercial whitelist
52             try:
53                 license_type = LicenseType(license_str)
54             except ValueError:
55                 license_type = LicenseType.UNKNOWN
56
57             # Group rows by set & rally
58             # Key: (set_num, rally_num) -> List of rows (dict)
59             rallies_data: Dict[Tuple[int, int], List[Dict[str, Any]]] = {}
60
61             with open(filepath, mode='r', encoding='utf-8') as f:
62                 reader = csv.DictReader(f)
63                 # Normalize column names to lowercase/strip
64                 reader.fieldnames = [name.lower().strip() for name in reader.fieldnames] if
reader.fieldnames else []
65
66                 for row in reader:
67                     # Resolve key columns (tolerate empty cells and float-strings)
68                     set_num = _safe_int(row.get('set'), 1)
69                     rally_num = _safe_int(row.get('rally'), 0)

```

```

70         ball_round = _safe_int(row.get('ball_round', row.get('ball_seq')), 1)
71
72         # Resolve time (safely parse float or HH:MM:SS strings)
73         raw_time = row.get('time', row.get('timestamp', row.get('sec', '0.0')))
74         time_val = parse_time_string(raw_time)
75
76         key = (set_num, rally_num)
77         if key not in rallies_data:
78             rallies_data[key] = []
79
80         rallies_data[key].append({
81             'ball_round': ball_round,
82             'time': time_val,
83             'type': row.get('type', ''),
84             'score_a': row.get('score_a', row.get('round_score_a', '0')),
85             'score_b': row.get('score_b', row.get('round_score_b', '0'))
86         })
87
88     # First pass: build raw rally records (unbuffered stroke bounds) in chrono
89     order.
90     raw_rallies: List[Dict[str, Any]] = []
91     for key in sorted(rallies_data.keys()):
92         rows_sorted = sorted(rallies_data[key], key=lambda x: x['ball_round'])
93         if not rows_sorted:
94             continue
95         first_stroke = rows_sorted[0]
96         last_stroke = rows_sorted[-1]
97         raw_rallies.append({
98             'first_time': first_stroke['time'],
99             'last_time': last_stroke['time'],
100             'ending_reason': last_stroke['type'] if last_stroke['type'] else None,
101             'shots_count': len(rows_sorted),
102             'score_before': f"{first_stroke['score_a']}-{first_stroke['score_b']}",
103             'score_after': f"{last_stroke['score_a']}-{last_stroke['score_b']}",
104         })
105
106     # Second pass: apply start/end buffers, then resolve any boundary where the
107     # padding of two adjacent rallies would collide by meeting at the midpoint of
108     # the gap between their nearest strokes. This keeps the buffers from ever
109     # producing an artificial overlap (which the validator would otherwise reject).
110     starts = [max(0.0, rr['first_time'] - self.start_buffer) for rr in raw_rallies]
111     ends = [rr['last_time'] + self.end_buffer for rr in raw_rallies]
112     for i in range(len(raw_rallies) - 1):
113         if ends[i] > starts[i + 1]:
114             midpoint = (raw_rallies[i]['last_time'] + raw_rallies[i +
115 1]['first_time']) / 2
116             ends[i] = min(ends[i], midpoint)
117             starts[i + 1] = max(starts[i + 1], midpoint)
118
119     rallies: List[BadmintonRally] = []
120     for i, rr in enumerate(raw_rallies):
121         start_time = starts[i]
122         end_time = ends[i]
123         # Guarantee a strictly positive duration even after clamping.
124         if end_time <= start_time:
125             end_time = start_time + 0.001
126
127         rallies.append(BadmintonRally(
128             video_id=video_id,
129             rally_number=i + 1,
130             start_time=start_time,
131             end_time=end_time,
132             score_before=rr['score_before'],
133             score_after=rr['score_after'],
134             shots_count=rr['shots_count'],

```

```

133             ending_reason=rr['ending_reason']
134         ))
135
136     # Populate VideoMetadata
137     video_metadata = VideoMetadata(
138         video_id=video_id,
139         sport=SportType.BADMINTON,
140         video_url=video_url,
141         local_path=kwargs.get("local_path", None),
142         license_type=license_type,
143         license_url=kwargs.get("license_url", None),
144         is_commercial=self.validator.check_license_commercial(license_type),
145         status=CurationStatus.STAGING,
146         match_name=match_name,
147         fps=kwargs.get("fps", 30.0),
148         resolution=kwargs.get("resolution", "1080p")
149     )
150     return video_metadata, rallies
151

```

22. assistant (2026-06-10T05:34:29.808Z)

Now let me look at the tests and run the suite.

23. user (2026-06-10T05:34:30.837Z)

```

tests\test_downloader.py
tests\test_local_import.py
tests\test_schemas.py
tests\test_validator.py
tests\test_cli.py
tests\test_config.py
tests\test_parser.py
tests\test_parsing.py
tests\test_db.py
tests\test_cvat_parser.py
tests\test_reporting.py
tests\test_reporting_faq.py
tests\test_normalize_annotations.py

```

24. user (2026-06-10T05:34:31.725Z)

Sports Data Collector Configuration

Path to the SQLite database

database_path: "./data/sports_metadata.db"

Storage configuration for downloaded videos

```

video_storage:
  # Folder where downloaded or local videos will be stored
  local_dir: "./data/videos"
  # Maximum allowed size for automatic downloads (in Megabytes)
  max_download_size_mb: 12000
  # Whether to attempt video downloads automatically during ingestion
  auto_download: true
  # Optional cap on download resolution height (e.g. 1080 to avoid pulling 4K).
  # Leave unset/null to always fetch the best resolution available.
  max_height: null

```

Whitelist of licenses considered safe for commercial use

commercial_licenses:

- "MIT"
- "Apache-2.0"
- "BSD"
- "CC0"
- "CC-BY"
- "Public-Domain"
- "Proprietary" # Used for custom user-added local videos

Ingestion limits

```
ingestion_limits:
  max_total_download_size_mb: 60000
  max_total_video_count: 10
```

Whitelist of all sports supported in the system

```
supported_sports:
```

- "badminton"
- "tennis"
- "tabletennis"
- "pickleball"
- "cricket"

```
[build-system]
```

```
requires = ["setuptools>=61.0.0"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
```

```
name = "sports-data-collector"
```

```
version = "0.1.0"
```

```
description = "A collector and curator for sports video datasets with sport-specific schemas and license checks."
```

```
readme = "README.md"
```

```
requires-python = ">=3.9"
```

```
dependencies = [
```

```
    "pydantic>=2.0.0",
```

```
    "pyyaml>=6.0",
```

```
    "tabulate>=0.9.0"
```

```
]
```

```
[project.optional-dependencies]
```

Video acquisition (`fetch` / crawler). Also requires the external binaries `ffprobe` (FFmpeg) and, for YouTube high-res, a JS runtime such as `deno` ? neither is pip-installable; see README "External requirements".

```
fetch = [
```

```
    "yt-dlp>=2024.1.0"
```

```
]
```

```
dev = [
```

```
    "pytest>=7.0.0",
```

```
    "pytest-cov>=4.0.0"
```

```
]
```

Per-video observability reports (SOFT dependency: ingestion degrades to a no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once the private repo is published, e.g.:

obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3

For local development install it editable from a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

```
report = [
```

```
    "obsreport>=0.1.3",
```

```
]
```

25. user (2026-06-10T05:34:42.152Z)

..... [58%]
..... [100%]
124 passed in 2.56s

26. user (2026-06-10T05:34:54.797Z)

..... [58%]
..... [100%]
===== tests coverage =====
_____ coverage: platform win32, python 3.13.13-final-0 _____

| Name | Stmts | Miss | Cover | Missing |
|----------------------------------|-------|------|-------|--|
| ----- | | | | |
| src\cli\curate.py | 567 | 180 | 68% | 35, 166, 185, 209, 263, 287-288, 329-334, 337-339, 361, 371-373, 377, 380-381, 386-387, 391-392, 398-401, 409, 425, 457, 469-471, 479-480, 484-485, 533, 535, 549-582, 595-656, 687-688, 690-691, 702-709, 727-738, 761, 773-774, 783-814, 817-818, 825-827, 835, 841, 948, 950, 960 |
| src\cli\normalize_annotations.py | 133 | 35 | 74% | 56-74, 83, 88, 91, 93-94, 103, 106, 132-134, 207-214, 218 |
| src\core\config.py | 15 | 0 | 100% | |
| src\core\db.py | 191 | 7 | 96% | 164, 212, 241, 268, 297, 325, 537 |
| src\core\downloader.py | 82 | 9 | 89% | 106-109, 127-128, 138-139, 180 |
| src\core\parsing.py | 35 | 2 | 94% | 44-45 |
| src\core\schemas\badminton.py | 19 | 0 | 100% | |
| src\core\schemas\base.py | 36 | 0 | 100% | |
| src\core\schemas\cricket.py | 20 | 0 | 100% | |
| src\core\schemas\pickleball.py | 19 | 0 | 100% | |
| src\core\schemas\tabletennis.py | 19 | 0 | 100% | |
| src\core\schemas\tennis.py | 18 | 0 | 100% | |
| src\core\validator.py | 88 | 5 | 94% | 46, 68, 89, 111, 134 |
| src\parsers\badminton\coachai.py | 62 | 4 | 94% | 43, 54-55, 93 |
| src\parsers\base_parser.py | 7 | 1 | 86% | 15 |
| src\parsers\cvat__init__.py | 0 | 0 | 100% | |
| src\parsers\cvat\rally_cvat.py | 251 | 37 | 85% | 148, 162-163, 177-178, 189-190, 198-202, 208, 214, 295, 339-340, 377-396, 416-417, 420-421, 432, 441, 449, 510 |
| src\reporting__init__.py | 24 | 2 | 92% | 36, 63 |
| src\reporting\adapter.py | 76 | 3 | 96% | 141, 164-165 |
| ----- | | | | |
| TOTAL | 1662 | 285 | 83% | |
| 124 passed in 3.28s | | | | |

27. assistant (2026-06-10T05:35:02.373Z)

All 124 tests pass (83% coverage). Now let me cross-check the indexer side of the contract.

28. user (2026-06-10T05:35:03.515Z)

/c/Users/avidu/Projects/badminton-highlight-indexer/:
CLAUDE.md
README.md
TEST_RUN_SUMMARY.md
__pycache__
backend
config.json
docs
output
pytest.ini
requirements.txt
run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py


```

run_regression.py
scratch
scripts
setup.py
tests

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/:
__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py

```

29. user (2026-06-10T05:35:04.583Z)

```

Found 10 files
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seq_proto.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\batch.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\export_wasb_segments.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\audio\e1_probe.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\annotations.py

```

30. user (2026-06-10T05:35:09.847Z)

```

1      """Batch evaluation across a collector export manifest.
2
3      Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4      sports-data-collector's `curate export`, (optionally) process each video through
5      a segmenter so detections land in the indexer DB, then score every video's
6      detected rallies against its ground-truth CSV and aggregate the results.
7
8      This module holds the *pure* pieces (path resolution, stage selection, metric
9      aggregation) so they can be unit-tested without touching video files, the GPU,
10     or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11     """
12
13     import os
14     from typing import Dict, List, Optional
15
16
17     # Segmenters that only ever populate the `final` play_segments table (no raw
18     # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19     _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22     def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23         """Resolve a manifest `local_path` to an absolute path.

```

```

24
25     The collector stores paths like ``./data/videos\\shuttle1.mp4`` relative
26     to its own repo root and with Windows separators. Normalise separators, strip
27     a leading ``./``, and resolve relative paths against `videos_root` (the
28     collector root). Returns None for a missing/empty path.
29     """
30     if not local_path:
31         return None
32     p = local_path.replace("\\", "/")
33     if p.startswith("./"):
34         p = p[2:]
35     # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36     # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37     # This keeps collector manifest paths working in tests and mixed environments.
38     if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39         return os.path.normpath(p)
40     return os.path.normpath(os.path.join(videos_root, p))
41
42
43 def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44     """Pick which prediction stages to score.
45
46     ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47     but only ``final`` for segmenters that don't emit candidates. An explicit
48     request ('candidates' | 'final' | 'both') overrides the heuristic.
49     """
50     if requested == "both":
51         return ["candidates", "final"]
52     if requested in ("candidates", "final"):
53         return [requested]
54     # auto
55     if segmenter in _FINAL_ONLY_SEGMENTERS:
56         return ["final"]
57     return ["candidates", "final"]
58
59
60 def _safe_div(num: float, den: float) -> float:
61     return num / den if den else 0.0
62
63
64 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65     """Aggregate per-video stage scores into corpus-level metrics.
66
67     `per_video` is a list of dicts shaped like `report_to_dict` output (each has
68     `stages[stage]["score"]` with tp/fp/fn/mean_iou/...). For each stage we
69     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70     a TP-weighted mean IoU, plus the macro mean of per-video F1.
71     """
72     out: Dict[str, dict] = {}
73     for stage in stages:
74         tp = fp = fn = 0
75         iou_weighted = 0.0
76         f1s: List[float] = []
77         n_videos = 0
78         for rep in per_video:
79             sr = rep.get("stages", {}).get(stage)
80             if not sr:
81                 continue
82             s = sr["score"]
83             tp += s["true_positives"]
84             fp += s["false_positives"]
85             fn += s["false_negatives"]
86             iou_weighted += s["mean_iou"] * s["true_positives"]
87             f1s.append(s["f1"])
88             n_videos += 1

```

```

89         precision = _safe_div(tp, tp + fp)
90         recall = _safe_div(tp, tp + fn)
91         out[stage] = {
92             "videos": n_videos,
93             "tp": tp, "fp": fp, "fn": fn,
94             "micro_precision": precision,
95             "micro_recall": recall,
96             "micro_f1": _safe_div(2 * precision * recall, precision + recall),
97             "macro_f1": _safe_div(sum(f1s), len(f1s)),
98             "mean_iou": _safe_div(iou_weighted, tp),
99         }
100     return out
101
102
103 def format_aggregate(agg: Dict[str, dict]) -> str:
104     """Human-readable corpus-level summary table."""
105     lines = [
106         "=" * 78,
107         " CORPUS AGGREGATE",
108         "=" * 78,
109         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
110         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
111         "-" * 78,
112     ]
113     for stage, a in agg.items():
114         lines.append(
115             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
116             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
117             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
118         )
119     lines.append("=" * 78)
120     return "\n".join(lines)
121

```

31. user (2026-06-10T05:35:14.969Z)

```

1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
12        collector/data/eval_export/manifest.json \
13        --segmenter motion --only shuttlest_1
14
15    Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
16        python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
17        --json output/phase3_report.json
18
19    Re-score only (videos already processed), no re-segmentation:
20        python run_phase3_eval.py --manifest ../manifest.json --score-only
21    """
22
23    import os
24    import sys
25    import json
26    import time
27    import argparse
28    import logging

```

```

29     from dotenv import load_dotenv
30     load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32     from backend.infrastructure.database import Database
33     from backend.config import load_config
34     from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35     # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36     from backend.pipeline.segmenters import segmenter_registry
37     from backend.eval.annotations import load_annotations
38     from backend.eval.harness import evaluate, format_report_text, report_to_dict
39     from backend.eval import batch
40
41     logger = logging.getLogger(__name__)
42
43
44     def build_parser() -> argparse.ArgumentParser:
45         p = argparse.ArgumentParser(
46             description="Batch process + evaluate a collector export manifest (Phase 3).",
47             formatter_class=argparse.RawDescriptionHelpFormatter,
48         )
49         p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
50         p.add_argument("--videos-root", default=None,
51             help="Root for resolving manifest relative local_paths "
52             "(default: the collector repo root, two levels above the
manifest).")
53         p.add_argument("--segmenter", default="motion",
54             help="Segmenter to run (default: motion ? free/CPU). "
55             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
56         p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
57             default="auto",
58             help="Which prediction stage(s) to score (default: auto by
segmenter).")
59         p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
60         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
61         p.add_argument("--only", action="append", default=None,
62             help="Only this video_id (repeatable).")
63         p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
64         p.add_argument("--max-frames", type=int, default=None,
65             help="Cap segmenter frames (fast validation).")
66         p.add_argument("--score-only", action="store_true",
67             help="Skip segmentation; only score detections already in the DB.")
68         p.add_argument("--reprocess", action="store_true",
69             help="Re-run segmentation even if the video already has segments.")
70         p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
71         p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
72         p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
73         return p
74
75     def main(argv=None) -> int:
76         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
77         args = build_parser().parse_args(argv)
78
79         manifest_path = os.path.abspath(args.manifest)
80         if not os.path.exists(manifest_path):
81             print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
82             return 2
83         manifest_dir = os.path.dirname(manifest_path)
84         videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))

```

```

85
86     with open(manifest_path) as f:
87         manifest = json.load(f)
88         entries = manifest.get("eval_sets", [])
89
90     cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
91     db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92     os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93     db = Database(db_path)
94
95     stages = batch.default_stages_for(args.segmenter, args.stage)
96
97     # Resolve the segmenter class up front (unless purely scoring).
98     segmenter = None
99     if not args.score_only:
100         seg_cls = segmenter_registry.get(args.segmenter)
101         if not seg_cls:
102             print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                   f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
104         return 2
105         segmenter = seg_cls(db, cfg)
106
107     # Filter entries.
108     targets = []
109     for e in entries:
110         if args.only and e["video_id"] not in args.only:
111             continue
112         targets.append(e)
113     if args.limit:
114         targets = targets[: args.limit]
115
116     per_video = []
117     skipped = []
118     print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
119
120     for e in targets:
121         vid = e["video_id"]
122         video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
123         csv_path = os.path.join(manifest_dir, e["csv"])
124
125         if not video_path or not os.path.exists(video_path):
126             skipped.append((vid, "no video file"))
127             print(f"[skip] {vid}: no video file ({e.get('local_path')})")
128             continue
129         if not os.path.exists(csv_path):
130             skipped.append((vid, "no GT csv"))
131             print(f"[skip] {vid}: GT csv missing ({csv_path})")
132             continue
133
134         video_id = get_file_md5(video_path)
135
136         # Segment unless score-only or already present (and not --reprocess).
137         if not args.score_only:
138             already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139             if already and not args.reprocess:
140                 print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141             else:
142                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                 db.add_video(video_id, video_path, "processed", [])
144                 t0 = time.time()

```

```

145         segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
146         print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
147
148         gt = load_annotations(csv_path)
149         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
150         print(format_report_text(report, show_failures=args.show_failures))
151         print()
152         d = report_to_dict(report)
153         d["collector_video_id"] = vid
154         per_video.append(d)
155
156         agg = batch.aggregate(per_video, stages)
157         print(batch.format_aggregate(agg))
158         if skipped:
159             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161         if args.json_out:
162             with open(args.json_out, "w") as f:
163                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
164                     "aggregate": agg, "per_video": per_video,
165                     "skipped": skipped}, f, indent=2)
166             print(f"\nJSON report written to {args.json_out}")
167
168         return 0
169
170
171 if __name__ == "__main__":
172     raise SystemExit(main())
173

```

32. user (2026-06-10T05:35:20.456Z)

```

1      """Load hand-annotated ground-truth rally timestamps from CSV.
2
3      Annotation file format (one file per video):
4
5          start,end
6          00:01:12,00:01:39
7          102.5,131.0
8
9      - A header row `start,end` is optional (auto-detected and skipped).
10     - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11       `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12     - Blank lines and `#` comment lines are ignored.
13     """
14
15     import csv
16     import logging
17     from typing import List, Tuple
18
19     logger = logging.getLogger(__name__)
20
21     Interval = Tuple[float, float]
22
23
24     def parse_timestamp(value: str) -> float:
25         """Parse a timestamp string to float seconds.
26
27         Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28         Raises ValueError on anything unparseable so bad annotations fail loudly.
29         """
30         value = value.strip()

```

```

31         if not value:
32             raise ValueError("empty timestamp")
33         if ":" in value:
34             parts = value.split(":")
35             if len(parts) > 3:
36                 raise ValueError(f"too many ':' segments in timestamp '{value}'")
37             total = 0.0
38             for part in parts:                # most-significant first (H, M, S)
39                 total = total * 60 + float(part)
40             return total
41         return float(value)
42
43
44     def load_annotations(csv_path: str) -> List[Interval]:
45         """Load and validate ground-truth rally intervals from a CSV file.
46
47         Returns intervals sorted by start time. Raises ValueError on malformed rows
48         (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49         than silently skewing scores.
50         """
51         intervals: List[Interval] = []
52         with open(csv_path, newline="") as f:
53             reader = csv.reader(f)
54             for lineno, row in enumerate(reader, start=1):
55                 if not row:
56                     continue
57                 first = row[0].strip()
58                 if not first or first.startswith("#"):
59                     continue
60                 # Skip an optional header row.
61                 if first.lower() in ("start", "start_time"):
62                     continue
63                 if len(row) < 2:
64                     raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65                 try:
66                     start = parse_timestamp(row[0])
67                     end = parse_timestamp(row[1])
68                 except ValueError as e:
69                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70                 if start >= end:
71                     raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72                 intervals.append((start, end))
73
74         intervals.sort(key=lambda iv: iv[0])
75         logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76         return intervals
77
78
79     def write_scaffold(csv_path: str) -> None:
80         """Write an empty annotation file with just the header, for hand-filling."""
81         with open(csv_path, "w", newline="") as f:
82             writer = csv.writer(f)
83             writer.writerow(["start", "end"])
84

```

33. user (2026-06-10T05:35:22.101Z)

```

29-Config = Tuple[float, float, float]    # (velocity_thresh, merge_gap, min_window_duration)
30-
31-BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
defaults
32-
33-
34:def load_golden_gts(csv_path: str) -> List[Interval]:

```

```

35:     """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36:     gts: List[Interval] = []
37:     with open(csv_path, newline="", encoding="utf-8") as f:
38:         reader = csv.DictReader(f)
39:         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or [])]
40:         for row in reader:
41:             row = {(k or "").strip().lower(): v for k, v in row.items()}
42:             try:
43:                 s = float((row.get("start_time") or "").strip())
44:                 e = float((row.get("end_time") or "").strip())
45:             except (TypeError, ValueError):
46:                 continue
47:             if e > s:
48:                 gts.append((s, e))
49:     return sorted(gts)
--
101-
102-
103-def main():
104-     ap = argparse.ArgumentParser()
105-     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106:     ap.add_argument("--labels", required=True, help="golden labels CSV (start_time/end_time
cols)")
107-     ap.add_argument("--fps", type=float, default=59.94)
108-     ap.add_argument("--frame-width", type=float, default=1920.0)
109-     ap.add_argument("--iou", type=float, default=0.5)
110-     args = ap.parse_args()
111-     run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)

```

34. user (2026-06-10T05:36:10.687Z)

```

1      # Roora Annotation Brief ? Rally Golden Set
2
3      **Project:** racket-sport rally segmentation golden / regression dataset
4      **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5      **Companion doc:** [ANNOTATION_GUIDE.md](./ANNOTATION_GUIDE.md)
6
7      > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8      > imports directly via `python -m src.cli.curate import-local`.
9
10     ## 0. What we need in one sentence
11     For each provided match video, produce **one CSV file** listing **every rally** with its
12     start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14     ## 1. Deliverable format (canonical)
15     One CSV per video (UTF-8), named exactly `

```



```

29 | `score_after` | optional, `"A-B"`. |
30 | `notes` | optional free text. **Required** when `ending_reason = other`. |
31
32 **Times (mm:ss ? decimal seconds).** Raters type only `start_mmss`/`end_mmss`. The
template
33 Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
row 2):
34
35 ```
36 =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
37 ```
38
39 On **Download as CSV**, the computed decimal seconds are exported. (If your tool already
40 shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
columns blank.)
41
42 Example (badminton):
43 ```csv
44 rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
_outcome,score_before,score_after,notes
45 1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
46 2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
47 3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
pressure)
48 ```
49
50 **Critical format rules**
51 - `start_time`/`end_time` that reach us **must be decimal seconds** (the formula
guarantees this).
52 - Times are relative to the **start of the file we provide**. Do **not** trim, clip, or
53 re-encode the video ? our pipeline identifies a video by its file checksum.
54 - `end_time` > `start_time` for every row.
55 - No overlaps: `end_time[N]` <= `start_time[N+1]`.
56 - Everything between one rally's `end_time` and the next rally's `start_time` is **dead
time** and gets **no row**.
57
58 ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
59 1. **Ignore all dead time.** Only label active play.
60 2. **Include every rally ? exclude none.** Every served point, including **service
faults**,
61 1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
project.
62 **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
63 like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
64 cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
65 in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
66 3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
67
68 ## 3. Scope & phasing
69 **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
tennis),
70 each < 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review
jointly and
71 lock the Guide before scale-up.
72
73 **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
total), each
74 ~30 min. Same format, rules, and template.
75

```

```

76 > **Doubles are included** across the program. Rally rules are identical to singles; the
77 > badminton- and pickleball-specific doubles notes are in the Guide.
78
79 ## 4. Quality bar (acceptance criteria)
80 Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81 ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82 - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83 - **Boundaries:** `start_time`/`end_time` within **±0.3 s** of true serve-contact /
landing.
84 - **shots_count:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
85 - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86 - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88 ## 5. Per-sport quick reference (full version + examples in the Guide)
89 **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle
floors / fault · shot: each racket?shuttle contact.
90 **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double
bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are **not** shots;
two-bounce rule is normal play).
91 **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal
return · shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
`let`.
92
93 ## 6. ending_reason values (decision tree in the Guide)
94 This vocabulary attributes **why** the point ended; it is shared across all
95 net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
96 the last shot physically landed in the separate **last_shot_outcome** column
97 (`in` \| `net` \| `out` \| `n_a`) ? e.g. a forced error that flew long is
98 `ending_reason=forced_error, last_shot_outcome=out`.
99
100 | value | meaning |
101 |---|---|
102 | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
winner). |
103 | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104 | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
time and position to make. |
105 | `service_fault` | the serve itself failed / was illegal. |
106 | `let` | the point was replayed. |
107 | `other` | anything else / cannot tell. **Must** add a note. |
108
109 ## 7. Recommended capture method
110 Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111 **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112 (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113 `seconds = frame_number / fps` (we provide fps per video).
114
115 ## 8. Logistics & contacts
116 - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117 - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118 - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119 - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120 - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121 - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**

```

```

122 - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124 ## 9. Pricing request (please quote)
125 1. Price **per rally** and **per minute** of footage.
126 2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127 3. Indicative **scale** price for ~36?45 longer videos.
128 4. Turnaround for the pilot and throughput (rallies/day) at scale.
129 5. QA process and rework policy.
130 6. Whether annotators have racket-sport familiarity (helpful, not required).
131 7. Confirmation you can deliver the CSV schema above.
132

```

35. user (2026-06-10T05:36:20.602Z)

```

1  """Tests for the CVAT rally adapter (src/parsers/cvat/rally_cvat.py).
2
3  The adapter's whole reason for existing is to (a) recompute rally time from the
4  box FRAME RANGE at the true fps ? ignoring the vendor's wrong start/end_time
5  attributes ? and (b) report content defects rather than silently fixing them.
6  These tests pin both behaviours, plus the end-to-end hand-off to import-local.
7  """
8  import csv as _csv
9
10 from src.parsers.cvat.rally_cvat import (
11     RallyRecord, parse_cvat_boxes, build_records, detect_issues, convert,
12     write_canonical_csv, load_canonical_csv, build_summary, render_validation_md,
13     write_review_csv,
14 )
15 from src.cli.curate import SportsDataCLI
16
17
18 # --- helpers -----
19
20 def _img_box(frame, attrs):
21     rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
22     return (f'<image id="{frame}" name="frame_{frame:06d}" width="1920" height="1080">'
23           f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
24
25
26 def _write_cvat(path, images, step=30, stop_frame=1000):
27     body = "".join(images)
28     xml = (
29         '<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
30         f'<meta><job><stop_frame>{stop_frame}</stop_frame>'
31         f'<frame_filter>step={step}</frame_filter></job></meta>'
32         f'{body}</annotations>'
33     )
34     path.write_text(xml, encoding="utf-8")
35     return str(path)
36
37
38 def _rec(rn, start, end, shots=5, reason="winner", fmin=None, fmax=None, n=3,
outcome=None):
39     return RallyRecord(
40         rally_number=rn, start_time=start, end_time=end, shots_count=shots,
41         ending_reason=reason, last_shot_outcome=outcome,
42         score_before=None, score_after=None, notes=None,
43         frame_min=fmin if fmin is not None else int(start * 60),
44         frame_max=fmax if fmax is not None else int(end * 60), n_boxes=n)
45
46
47 # --- the core fix: time from frames, vendor times ignored -----
48
49 def test_time_recomputed_from_frames_not_attributes(tmp_path):
50     # rally 1 spans source frames 60..120; the vendor's start/end attributes are

```

```

51 # deliberately wrong (the frame/900 bug). At 60 fps the truth is 1.0..2.0s.
52 xml = _write_cvat(tmp_path / "j.xml", [
53     _img_box(60, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4",
54                 "shots_count": "6.0", "ending_reason": "winner",
55                 "score_before (A-B)": "0-0", "notes": "clean"}),
56     _img_box(90, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
57     _img_box(120, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
58 ])
59 res = convert(xml, fps=60.0)
60 assert len(res.records) == 1
61 r = res.records[0]
62 assert r.start_time == 1.0 and r.end_time == 2.0      # frames/fps, not 0.2/1.4
63 assert r.vendor_start_time == "0.2"                  # preserved for transparency
64 assert r.shots_count == 6
65 assert r.ending_reason == "winner"
66 assert r.score_before == "0-0"                       # "(A-B)" alias resolved
67 assert r.notes == "clean"
68
69
70 def test_step_and_fps_read_from_meta_and_arg(tmp_path):
71     xml = _write_cvat(tmp_path / "j.xml", [
72         _img_box(0, {"rally_number": "1", "shots_count": "3"}),
73         _img_box(30, {"rally_number": "1"}),
74     ], step=30)
75     res = convert(xml, fps=60.0)
76     assert res.step == 30
77     assert res.fps == 60.0
78
79
80 def test_fps_required_when_unavailable(tmp_path):
81     xml = _write_cvat(tmp_path / "j.xml", [_img_box(0, {"rally_number": "1"})])
82     try:
83         convert(xml) # no fps, no video_path
84     except ValueError as e:
85         assert "fps" in str(e).lower()
86     else:
87         raise AssertionError("expected ValueError when fps cannot be determined")
88
89
90 # --- track-mode parsing + outside filtering -----
91
92 def test_track_mode_and_outside_filtering(tmp_path):
93     xml = (
94         '<?xml version="1.0"?><annotations><meta><job>'
95         '<stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter></job></meta>'
96         '<track id="0" label="rally">'
97         '<box frame="60" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
98         '<attribute name="rally_number">1</attribute>'
99         '<attribute name="shots_count">4</attribute></box>'
100         '<box frame="120" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
101         '<attribute name="rally_number">1</attribute></box>'
102         '<box frame="150" outside="1" xtl="1" ytl="1" xbr="2" ybr="2">' # terminator
103         '<attribute name="rally_number">1</attribute></box>'
104         '</track></annotations>'
105     )
106     p = tmp_path / "track.xml"
107     p.write_text(xml, encoding="utf-8")
108     meta, boxes = parse_cvat_boxes(str(p))
109     assert len(boxes) == 2      # the outside=1 terminator dropped
110     recs = build_records(boxes, fps=60.0)
111     assert recs[0].frame_max == 120      # not 150 (the terminator)
112     assert recs[0].end_time == 2.0
113
114
115 # --- issue detection: surfaced, not silently fixed -----

```

```

116
117 def test_overlap_is_a_blocker():
118     recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0)] # r2 starts before r1 ends
119     issues = detect_issues(recs)
120     overlaps = [i for i in issues if i.code == "overlap"]
121     assert len(overlaps) == 1 and overlaps[0].severity == "blocker"
122
123
124 def test_zero_duration_is_a_blocker():
125     recs = [_rec(1, 2.0, 2.0, fmin=120, fmax=120, n=1)]
126     issues = detect_issues(recs)
127     assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)
128
129
130 def test_offvocab_ending_reason_flagged():
131     recs = [_rec(1, 1.0, 3.0, reason="into_net")]
132     issues = detect_issues(recs)
133     assert any(i.code == "ending_reason_vocab" for i in issues)
134     # the adopted forced/unforced terms are accepted, not flagged
135     ok = detect_issues([_rec(1, 1.0, 3.0, reason="forced_error")])
136     assert not any(i.code == "ending_reason_vocab" for i in ok)
137
138
139 def test_low_shot_density_flags_inflated_span():
140     recs = [_rec(1, 0.0, 16.0, shots=2)] # 2 shots over 16s -> 0.12/s
141     issues = detect_issues(recs)
142     assert any(i.code == "shot_density" for i in issues)
143
144
145 def test_outcome_vocab_flagged():
146     bad = detect_issues([_rec(1, 1.0, 3.0, outcome="long")])
147     assert any(i.code == "outcome_vocab" for i in bad)
148     # the agreed outcomes are accepted, not flagged
149     ok = detect_issues([_rec(1, 1.0, 3.0, outcome="out")])
150     assert not any(i.code == "outcome_vocab" for i in ok)
151
152
153 # --- end-to-end: emitted CSV imports via the existing import-local parser ---
154
155 def test_canonical_csv_round_trips_through_import_local(tmp_path):
156     xml = _write_cvat(tmp_path / "j.xml", [
157         _img_box(60, {"rally_number": "1", "start_time": "0.2", "end_time": "1.4",
158             "shots_count": "6", "ending_reason": "winner",
159             "last_shot_outcome": "in"}),
160         _img_box(120, {"rally_number": "1"}),
161         _img_box(180, {"rally_number": "2", "shots_count": "4",
162             "ending_reason": "forced_error", "last_shot_outcome": "out"}),
163         _img_box(240, {"rally_number": "2"}),
164     ])
165     res = convert(xml, fps=60.0)
166     assert res.n_blockers == 0
167     assert res.records[1].last_shot_outcome == "out" # parsed from the box attribute
168     out = tmp_path / "canonical.csv"
169     write_canonical_csv(res.records, str(out))
170
171     # Re-read with the actual import-local parser (self is unused -> None is safe).
172     items = SportsDataCLI._parse_local_annotations(None, "badminton", "vid1", str(out))
173     assert len(items) == 2
174     assert items[0].start_time == 1.0 and items[0].end_time == 2.0 # not 0.2/1.4
175     assert items[1].ending_reason == "forced_error"
176     assert items[1].last_shot_outcome == "out" # survives the round-trip into
the schema
177     assert items[1].start_time == 3.0 and items[1].end_time == 4.0
178
179

```

```

180 # --- validate-delivery: gap heuristic, CSV loading, report artifacts -----
181
182 def test_possible_missed_rally_gap_flagged():
183     # 20s of unlabeled time between two rallies -> flag (catches the 1:46-style miss)
184     recs = [_rec(1, 95.6, 98.0), _rec(2, 118.0, 119.6)]
185     issues = detect_issues(recs)
186     assert any(i.code == "possible_missed_rally" for i in issues)
187
188
189 def test_normal_gap_not_flagged():
190     recs = [_rec(1, 10.0, 14.0), _rec(2, 22.0, 26.0)] # 8s gap -> fine
191     issues = detect_issues(recs)
192     assert not any(i.code == "possible_missed_rally" for i in issues)
193
194
195 def test_load_canonical_csv_decimal_and_mmss(tmp_path):
196     p = tmp_path / "c.csv"
197     p.write_text(
198         "rally_number,start_time,end_time,start_mmss,end_mmss,shots_count,ending_reason,
199         last_shot_outcome\n"
200         "1,3.0,21.0,,,15,unforced_error,out\n"
201         "2,,,0:29.000,0:32.000,2,winner,in\n", encoding="utf-8")
202     recs = load_canonical_csv(str(p))
203     assert len(recs) == 2
204     assert recs[0].start_time == 3.0 and recs[0].last_shot_outcome == "out"
205     assert recs[1].start_time == 29.0 and recs[1].end_time == 32.0 # mm:ss fallback
206
207
208 def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
209     # Regression: a rally that legitimately starts at 0.0s must not be dropped.
210     # The old `start_time or start_mmss` fallback treated the falsy 0.0 as "missing"
211     # and skipped the row when there was no start_mmss column to fall back to.
212     p = tmp_path / "c.csv"
213     p.write_text(
214         "rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"
215         "1,0.0,4.0,8,winner,in\n"
216         "2,10.0,14.0,6,forced_error,out\n"
217         "3,20.0,24.0,5,unforced_error,out\n", encoding="utf-8")
218     recs = load_canonical_csv(str(p))
219     assert len(recs) == 3 # the zero-start row 1 is not lost
220     assert recs[0].start_time == 0.0 and recs[0].end_time == 4.0
221
222
223 def test_validation_artifacts(tmp_path):
224     recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0, reason="into_net")] # overlap + bad
225     vocab
226     issues = detect_issues(recs)
227     summary = build_summary(recs, issues, source="vid", fps=59.94)
228     assert summary["n_rallies"] == 2
229     assert summary["passed"] is False and summary["n_blockers"] >= 1
230     assert 2 in summary["flagged_rallies"]
231     md = render_validation_md(recs, issues, "vid")
232     assert "BLOCKED" in md and "## BLOCKER" in md
233     out = tmp_path / "review.csv"
234     write_review_csv(recs, issues, str(out))
235     rows = list(_csv.DictReader(open(out, encoding="utf-8")))
236     assert "VERIFIED(Y/N)" in rows[0] and "auto_flags" in rows[0]
237     assert any("overlap" in r["auto_flags"] for r in rows)

```

36. user (2026-06-10T05:36:28.894Z)

No matches found

37. user (2026-06-10T05:36:30.187Z)

```

3-The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4-ending_reason,sport). While a video is only partially tagged, the labels cover a
5-PREFIX of the video ? every rally up to some point is marked, and everything after is
6-simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7-the detector's *correct* rallies in the un-labeled tail would be counted as false
8-positives and crush precision.
9-
10-This harness fixes that by restricting the evaluation to the LABELED WINDOW
11:[window_start, window_end] (window_end defaults to the last labeled rally's end) and
12-dropping predicted segments that fall outside it. Within that window the human labels
13-ARE complete, so precision/recall/F1 are honest.
14-
15-It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16- - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17- - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18- - eval.metrics.match_intervals / score
19- - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
--
39-Interval = Tuple[float, float]
40-Config = Tuple[float, float, float]
41-
42-# Operating points to compare. The tuned config wins on its fit-video but over-segments
43-# and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44-TUNED: Config = (0.05, 1.0, 1.0)
45-
46-
47:def restrict_to_window(intervals: List[Interval], w0: float, w1: float) -> List[Interval]:
48-    """Keep intervals that overlap the labeled window [w0, w1].
49-
50-    A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51-    in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52-    they are neither rewarded nor penalized. GTs are assumed already inside the window.
53-    """
54-    return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55-
--
61-    if gate_crossings > 0 and points:
62-        preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63-        preds = restrict_to_window(preds, *window)
64-        sc = score(preds, gts, iou)
65-        return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc, "preds":
preds}
66-
67-
68-def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
69-        window_start: float = 0.0, window_end: Optional[float] = None,
70-        gate_crossings: int = 2, iou: float = 0.5,
71-        comparison_out: Optional[str] = None) -> dict:
72-    gts_all = load_golden_gts(labels_csv)
73-    if not gts_all:
74-        raise SystemExit(f"no usable human rallies in {labels_csv}")
75-    if window_end is None:
76-        window_end = max(e for _, e in gts_all)
77-    window = (window_start, window_end)
78-    gts = restrict_to_window(gts_all, *window)
79-
80-    predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
81-    points = TrackNetRunner.parse_trajectory_csv(trajjectory_csv)
82-    traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83-
84-    print("=== Pipeline vs PARTIAL human labels ===")

```

[Showing results with pagination = limit: 60]

```

1      """Best-quality video acquisition.
2
3      A single place that knows how to fetch the *highest resolution available* for a
4      source URL and report what was actually obtained. Both the ShuttleSet crawler
5      (initial ingestion) and the `fetch` CLI command (in-place upgrade of already-
6      ingested videos) go through here, so the resolution policy lives in exactly one
7      spot.
8
9      Design notes:
10     - `bv*+ba/b` selects best video + best audio, falling back to the best
11       pre-muxed stream. Combined with `-S res,fps,...`, yt-dlp sorts candidate
12       formats by resolution (then fps) descending and picks the top ? i.e. *always*
13       the best resolution available, regardless of codec/container quirks.
14     - `--merge-output-format mp4` guarantees an MP4 container, which the downstream
15       indexer requires.
16     - We never trust the requested resolution; we ffprobe the resulting file and
17       record the *actual* width/height/fps as the source of truth.
18     """
19
20     import os
21     import shutil
22     import subprocess
23     from dataclasses import dataclass
24     from typing import List, Optional, Tuple
25
26     # Best video+audio, prefer mp4/m4a, fall back to best single stream.
27     _FORMAT = "bv*+ba/b"
28     # Sort candidates so the highest resolution (then fps) wins; prefer mp4/m4a on ties.
29     _SORT = "res,fps,ext:mp4:m4a"
30
31
32     def height_to_label(height: Optional[int]) -> Optional[str]:
33         """Map a pixel height to a conventional resolution label (e.g. 1080 -> '1080p').
34
35         Snaps to the nearest standard tier at or below the actual height so a 1072px
36         crop still reads as '1080p' rather than an odd number.
37         """
38         if not height or height <= 0:
39             return None
40         # Allow a 5% tolerance below each tier so a lightly letterboxed/cropped frame
41         # (e.g. 1072px) still reads as the intended tier ('1080p') rather than dropping.
42         for tier in (2160, 1440, 1080, 720, 480, 360, 240, 144):
43             if height >= tier * 0.95:
44                 return f"{tier}p"
45         return f"{height}p"
46
47
48     def build_ytdlp_command(url: str, dest_path: str, max_height: Optional[int] = None,
49                             cookies_from_browser: Optional[str] = None,
50                             cookies_file: Optional[str] = None,
51                             remote_components: Optional[str] = "ejs:github",
52                             player_client: Optional[str] = "default") -> List[str]:
53         """Build the yt-dlp argv for a best-quality download.
54
55         `max_height`, when set, caps resolution (e.g. avoid pulling 4K); left unset
56         it fetches the absolute best available. Pure/argv-only so it is unit-testable
57         without invoking the network.
58
59         `player_client` selects YouTube's player client (default `default` = yt-dlp's
60         maintained client set, which serves real downloadable 1080p DASH with auth
61         cookies). Avoid `web`/`web_safari`: YouTube forces those into SABR streaming,
62         which strips download URLs and silently drops to the lowest stream (the 144p
63         bug). Harmless for non-YouTube URLs (the extractor-arg is ignored). Set to
64         None to omit the arg entirely.
65

```



```

66     `remote_components` enables yt-dlp's EJS challenge solver (default
67     `ejs:github`), the JS-runtime (deno) path for solving YouTube's n-challenge.
68     Set to None to disable (e.g. for non-YouTube sources or offline tests).
69     """
70     fmt = _FORMAT
71     if max_height:
72         # Cap each stream selector; keep the pre-muxed fallback capped too.
73         fmt = f"bv*[height<={max_height}]+ba/b[height<={max_height}]/b"
74     cmd = [
75         "yt-dlp",
76         "-f", fmt,
77         "-S", _SORT,
78         "--merge-output-format", "mp4",
79         "--no-playlist",
80         "--no-progress",
81         "-o", dest_path,
82     ]
83     if player_client:
84         cmd += ["--extractor-args", f"youtube:player_client={player_client}"]
85     if remote_components:
86         cmd += ["--remote-components", remote_components]
87     if cookies_from_browser:
88         cmd += ["--cookies-from-browser", cookies_from_browser]
89     if cookies_file:
90         cmd += ["--cookies", cookies_file]
91     cmd.append(url)
92     return cmd
93
94
95     def _env_with_deno_on_path() -> dict:
96         """Return an environment with the default deno install dir on PATH.
97
98         The EJS challenge solver needs a JS runtime (deno). The official installer
99         drops it at ~/.deno/bin, but a shell started before install won't have it on
100        PATH yet. We add it here so `fetch` works without requiring a terminal
101        restart ? without clobbering a deno the user already has elsewhere.
102        """
103        env = os.environ.copy()
104        if shutil.which("deno"):
105            return env # already discoverable
106        deno_bin = os.path.join(os.path.expanduser("~"), ".deno", "bin")
107        if os.path.isdir(deno_bin):
108            env["PATH"] = deno_bin + os.pathsep + env.get("PATH", "")
109        return env
110
111
112     def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int],
Optional[float]]:
113         """Return (width, height, fps) of a video file via ffprobe, best-effort.
114
115         Returns (None, None, None) if ffprobe is unavailable or the file can't be
116         parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117         """
118         if not shutil.which("ffprobe") or not os.path.exists(path):
119             return (None, None, None)
120         try:
121             out = subprocess.run(
122                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
123                  "-show_entries", "stream=width,height,avg_frame_rate",
124                  "-of", "default=noprint_wrappers=1:nokey=1", path],
125                 stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126             ).stdout.split()
127         except (subprocess.SubprocessError, OSError):
128             return (None, None, None)
129

```

```

130     width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131     height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132     fps = None
133     if len(out) > 2 and "/" in out[2]:
134         num, den = out[2].split("/")
135         try:
136             den_f = float(den)
137             fps = round(float(num) / den_f, 3) if den_f else None
138         except ValueError:
139             fps = None
140     return (width, height, fps)
141
142
143 @dataclass
144 class DownloadResult:
145     success: bool
146     path: Optional[str] = None
147     size_mb: float = 0.0
148     width: Optional[int] = None
149     height: Optional[int] = None
150     fps: Optional[float] = None
151     resolution_label: Optional[str] = None
152     error: Optional[str] = None
153
154
155 def download_best_video(url: str, dest_path: str, max_height: Optional[int] = None,
156                        max_size_mb: Optional[float] = None,
157                        cookies_from_browser: Optional[str] = None,
158                        cookies_file: Optional[str] = None,
159                        player_client: Optional[str] = "default") -> DownloadResult:
160     """Download the best-resolution video for `url` to `dest_path` (MP4).
161
162     On success, ffprobes the file and reports actual width/height/fps. If
163     `max_size_mb` is exceeded the file is removed and a failed result returned,
164     so callers never register an over-budget download.
165     """
166     os.makedirs(os.path.dirname(os.path.abspath(dest_path)), exist_ok=True)
167     cmd = build_ytdlp_command(url, dest_path, max_height=max_height,
168                              cookies_from_browser=cookies_from_browser,
169                              cookies_file=cookies_file,
170                              player_client=player_client)
171     try:
172         subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
173                        text=True, check=True, env=_env_with_deno_on_path())
174     except FileNotFoundError:
175         return DownloadResult(success=False, error="yt-dlp not found on PATH")
176     except subprocess.CalledProcessError as e:
177         return DownloadResult(success=False, error=(e.stderr or str(e)).strip()[:500])
178
179     if not os.path.exists(dest_path):
180         return DownloadResult(success=False, error="yt-dlp reported success but no file
was produced")
181
182     size_mb = os.path.getsize(dest_path) / (1024 * 1024)
183     if max_size_mb is not None and size_mb > max_size_mb:
184         os.remove(dest_path)
185         return DownloadResult(
186             success=False, size_mb=size_mb,
187             error=f"downloaded size {size_mb:.1f} MB exceeds limit {max_size_mb:.1f}
MB",
188         )
189
190     width, height, fps = probe_resolution_fps(dest_path)
191     return DownloadResult(
192         success=True, path=dest_path, size_mb=size_mb,

```

```

193         width=width, height=height, fps=fps,
194         resolution_label=height_to_label(height),
195     )
196

```

39. user (2026-06-10T05:37:56.129Z)

```

eval_export
mock_local_badminton_anno.json
roora_badminton_canonical.csv
roora_badminton_issues.txt
sports_metadata.db
staging_coachai_match.csv
temp
validation
videos
#!/usr/bin/env python3
import os
import sys
import shutil
import argparse
import math

from src.core.config import load_config

def get_file_or_dir_size(path):
    """Calculates the size of a file or directory in bytes."""
    if not os.path.exists(path):
        return 0
    if os.path.isfile(path):
        return os.path.getsize(path)

    total_size = 0
    for dirpath, _, filenames in os.walk(path):
        for f in filenames:
            fp = os.path.join(dirpath, f)
            if os.path.exists(fp) and not os.path.islink(fp):
                total_size += os.path.getsize(fp)
    return total_size

def format_size(size_bytes):
    """Formats bytes to a human-readable string."""
    if size_bytes == 0:
        return "0 B"
    size_name = ("B", "KB", "MB", "GB")
    i = int(math.floor(math.log(size_bytes, 1024)))
    p = math.pow(1024, i)
    s = round(size_bytes / p, 2)
    return f"{s} {size_name[i]}"

def get_all_items_in_dir(path):
    """Returns a list of absolute paths of all files/directories inside a directory."""
    items = []
    if not os.path.exists(path) or not os.path.isdir(path):
        return items
    for entry in os.listdir(path):
        items.append(os.path.join(path, entry))
    return items

def main():
    parser = argparse.ArgumentParser(
        description="Purge stored state including SQLite database, downloaded videos, and temporary files."
    )
    parser.add_argument(

```

```

    "--config",
    default="config/config.yaml",
320 /c/Users/avidu/Projects/sports-data-collector/src/crawlers/badminton_crawler.py
301 /c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py
 25 /c/Users/avidu/Projects/sports-data-collector/tests/test_config.py
235 /c/Users/avidu/Projects/sports-data-collector/tests/test_cvat_parser.py
247 /c/Users/avidu/Projects/sports-data-collector/tests/test_db.py
145 /c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py
 46 /c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py
111 /c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py
111 /c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py
 31 /c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py
135 /c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py
 49 /c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py
 37 /c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py
117 /c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py
1910 total

```

40. assistant (2026-06-10T05:38:07.052Z)

Now let me empirically verify the key landmines (duplicate rally_number PK collision, corrupt XML handling) in a temp sandbox.

41. user (2026-06-10T05:38:16.187Z)

```

INSERT raised: IntegrityError UNIQUE constraint failed: badminton_rallies.video_id,
badminton_rallies.rally_number
rows in table after failure: 0
validator ok? True errors: []
corrupt xml raises: ParseError | is ValueError? False | is ParseError? True

```

42. user (2026-06-10T05:38:42.842Z)

```

records: 0 issues: 0 blockers: 0
summary passed? True n_rallies: 0
Rally_Number (case only) records: 1

```

43. user (2026-06-10T05:38:54.615Z)

```

/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:58:def
test_import_local_badminton_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:68:def
test_import_local_pickleball_csv(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:76:def
test_import_local_tabletennis_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:85:def
test_import_local_tennis_csv(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:93:def
test_import_local_cricket_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:106:def
test_import_local_unsupported_sport_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:113:def
test_import_local_missing_annotations_exits(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:118:def
test_import_local_overlap_fails_validation(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:126:def
test_import_local_unknown_license_defaults(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:135:def
test_import_local_json_missing_required_field_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:143:def
test_import_local_bad_extension_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:154:def
test_status_empty_and_populated(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:162:def

```

```
test_export_eval_set(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:197:def test_curate_quit(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:203:def test_curate_empty(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:207:def
test_curate_review_keep(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:214:def
test_curate_approve(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:221:def test_curate_reject(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:228:def
test_curate_play_segment(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:250:def
test_fetch_downloads_and_updates_db(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:261:def
test_fetch_handles_failure(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:283:def test_main_dispatch(argv,
method):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:290:def
test_main_import_local_dispatch():
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:298:def
test_main_no_command_prints_help():
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:16:def
test_insert_and_get_video(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:39:def
test_badminton_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:62:def
test_reinsert_video_preserves_child_rallies(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:86:def
test_get_segment_intervals_badminton(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:104:def
test_pickleball_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:128:def
test_tabletennis_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:149:def
test_get_segment_intervals_pickleball(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:163:def
test_tennis_point_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:180:def
test_cricket_delivery_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:198:def
test_delete_cascades_child_rows(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:211:def
test_get_segment_intervals_empty_returns_empty(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:216:def
test_get_status_counts(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:230:def
test_update_status_and_delete(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py:17:def
test_parse_pickleball_csv(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py:34:def
test_parse_tabletennis_json(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:18:def
test_drops_blank_and_reversed_end_rows():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:32:def
test_drops_negative_start_rows():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:43:def
test_renumbers_fractional_and_duplicate_ids_by_start():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:56:def
test_duplicate_truncating_ids_are_separated():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:66:def
test_mmss_times_converted_and_counted():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:74:def
test_pipe_delimited_reason_folds_to_primary_plus_note():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:82:def
test_clean_lean_input_is_passthrough():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:91:def
```

```
test_normalized_output_ingests_through_import_local_parser(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:15:def
test_license_checks(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:23:def
test_validate_badminton_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:43:def
test_validate_tennis_points(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:61:def
test_validate_empty_lists_are_ok(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:69:def
test_validate_cricket_deliveries(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:85:def
test_validate_pickleball_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:102:def
test_validate_tabletennis_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:15:def
test_rally_end_not_after_start_raises(cls):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:21:def
test_rally_negative_start_raises(cls):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:26:def
test_tennis_point_guards():
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:33:def
test_cricket_delivery_guards():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:15:def
test_command_requests_best_quality_mp4():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:29:def
test_command_can_disable_remote_components():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:34:def
test_command_pins_player_client_by_default():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:40:def
test_command_player_client_can_be_disabled():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:45:def
test_command_caps_resolution_when_max_height_set():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:51:def
test_command_passes_cookies_when_requested():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:56:def
test_command_passes_cookies_file():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:67:def
test_height_to_label(height, label):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:73:def
test_probe_resolution_fps_pares_ffprobe(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:86:def
test_probe_resolution_fps_missing_ffprobe(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:100:def
test_download_best_video_success(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:114:def
test_download_best_video_over_size_limit_removes_file(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:128:def
test_download_best_video_ytdlp_missing(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:138:def
test_download_best_video_ytdlp_error(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:20:def
test_coachai_parser(mock_coachai_csv):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:56:def
test_coachai_parser_time_string():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:65:def
test_coachai_parser_tolerates_empty_cells(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:81:def
test_coachai_buffers_never_create_overlap(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:99:def
test_coachai_commercial_flag_uses_config_whitelist(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:26:def
test_issue_to_flag_severity_and_customer_message():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:39:def
test_delivery_with_blocker_fails_and_surfaces_to_customer():
```

```

/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:60:def
test_delivery_flag_carries_frame_evidence_and_review_framing():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:75:def
test_delivery_clean_passes():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:91:def
test_import_clean_run():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:102:def
test_import_noncommercial_warns():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:108:def
test_import_missing_fps_warns():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:116:def
test_emit_delivery_writes_into_out_dir(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:125:def
test_emit_import_writes_next_to_video(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py:37:def
test_all_collector_faq_refs_resolve():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py:46:def
test_actionable_rules_have_faq_refs():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:6:def test_safe_int():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:16:def
test_safe_float_required():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:25:def
test_parse_time_string():
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:6:def
test_load_config_missing_returns_empty(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:10:def
test_load_config_reads_values(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:18:def
test_load_config_malformed_returns_empty(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:24:def
test_get_database_path_default():

```

44. user (2026-06-10T05:39:08.463Z)

```

1      """End-to-end coverage for the SportsDataCLI commands.
2
3      Each test runs against a throwaway config + SQLite DB in tmp_path, so the real
4      data/ directory is never touched. Network (yt-dlp) and the browser are mocked.
5      """
6      import csv as _csv
7      import json
8      from types import SimpleNamespace
9      from unittest import mock
10
11     import pytest
12     import yaml
13
14     from src.cli.curate import SportsDataCLI
15     from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
16     from src.core.schemas.badminton import BadmintonRally
17
18
19     @pytest.fixture
20     def cli(tmp_path):
21         cfg = {
22             "database_path": str(tmp_path / "test.db"),
23             "video_storage": {
24                 "local_dir": str(tmp_path / "videos"),
25                 "max_download_size_mb": 100,
26                 "auto_download": False,
27                 "max_height": None,
28             },
29             "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-
Domain", "Proprietary"],
30             "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball",

```

```

"cricket"],
31     }
32     cfg_path = tmp_path / "config.yaml"
33     cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
34     return SportsDataCLI(config_path=str(cfg_path))
35
36
37     def _import_args(**kw):
38         base = dict(
39             sport=None, video_path="nonexistent.mp4", annotations_path=None,
40             match_name=None, video_id=None, license="Proprietary", fps=30.0,
41             resolution="1080p",
42             )
43         base.update(kw)
44         return SimpleNamespace(**base)
45
46     def _write_csv(path, header, rows):
47         with open(path, "w", newline="", encoding="utf-8") as f:
48             w = _csv.writer(f)
49             w.writerow(header)
50             for r in rows:
51                 w.writerow(r)
52
53
54     # ----- #
55     # import-local: every sport, both JSON and CSV (covers dispatch + parse)      #
56     # ----- #
57
58     def test_import_local_badminton_json(cli, tmp_path):
59         p = tmp_path / "bad.json"
60         p.write_text(json.dumps([

```

45. user (2026-06-10T05:39:08.891Z)

```

154     def test_status_empty_and_populated(cli, tmp_path):
155         cli.status() # empty branch
156         p = tmp_path / "b.json"
157         p.write_text(json.dumps([{"rally_number": 1, "start_time": 1.0, "end_time": 2.0}]),
158             encoding="utf-8")
159         cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
160             video_id="s1"))
161         cli.status() # populated branch
162
163     def test_export_eval_set(cli, tmp_path):
164         p = tmp_path / "b.json"
165         p.write_text(json.dumps([
166             {"rally_number": 1, "start_time": 10.0, "end_time": 20.0},
167             {"rally_number": 2, "start_time": 25.0, "end_time": 35.0},
168         ]), encoding="utf-8")
169         cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
170             video_id="exp1", license="MIT"))
171
172         out_dir = tmp_path / "out"
173         args = SimpleNamespace(out_dir=str(out_dir), sport=None, include_staging=False,
174             include_noncommercial=False)
175         cli.export_eval_set(args)
176
177         manifest = out_dir / "manifest.json"
178         assert manifest.exists()
179         doc = json.loads(manifest.read_text(encoding="utf-8"))
180         assert doc["total_videos"] == 1
181         assert doc["total_segments"] == 2
182         assert (out_dir / "badminton" / "exp1.csv").exists()

```



```

180
181
182 # ----- #
183 # interactive curate loop (mocked input + browser) #
184 # ----- #
185
186 def _stage_badminton(cli, vid="stg1"):
187     cli.db.insert_video(VideoMetadata(
188         video_id=vid, sport=SportType.BADMINTON, license_type=LicenseType.MIT,
189         is_commercial=True, match_name="Staged Match", status=CurationStatus.STAGING,
190         video_url="https://www.youtube.com/watch?v=abc123",
191     ))
192     cli.db.insert_badminton_rallies([
193         BadmintonRally(video_id=vid, rally_number=1, start_time=10.0, end_time=20.0,
shots_count=5),
194     ])
195
196
197 def test_curate_quit(cli):
198     _stage_badminton(cli)
199     with mock.patch("builtins.input", side_effect=["q"]):
200         cli.curate()
201
202
203 def test_curate_empty(cli):
204     cli.curate() # no staging entries branch
205
206
207 def test_curate_review_keep(cli):
208     _stage_badminton(cli)
209     with mock.patch("builtins.input", side_effect=["1", "k"]):
210         cli.curate()
211     assert cli.db.get_video("stg1").status == CurationStatus.STAGING
212
213
214 def test_curate_approve(cli):
215     _stage_badminton(cli)
216     with mock.patch("builtins.input", side_effect=["1", "a"]):
217         cli.curate()
218     assert cli.db.get_video("stg1").status == CurationStatus.CURATED
219
220
221 def test_curate_reject(cli):
222     _stage_badminton(cli)
223     with mock.patch("builtins.input", side_effect=["1", "r"]):
224         cli.curate()
225     assert cli.db.get_video("stg1") is None
226
227
228 def test_curate_play_segment(cli):
229     _stage_badminton(cli)
230     with mock.patch("builtins.input", side_effect=["1", "p", "1", "k"]), \
231         mock.patch("webbrowser.open") as wb:
232         cli.curate()
233     wb.assert_called_once()
234
235
236 # ----- #
237 # fetch (mocked downloader) #
238 # ----- #
239
240 def _fetch_args(**kw):
241     base = dict(
242         sport=None, video_id=None, status="all", min_height=720, max_height=None,
243         force=False, include_noncommercial=False, cookies_from_browser=None,

```

```

244         cookies=None, player_client="default", sleep=0.0, retries=0, retry_backoff=0.0,
245     )
246     base.update(kw)
247     return SimpleNamespace(**base)
248
249
250     def test_fetch_downloads_and_updates_db(cli, tmp_path):
251         _stage_badminton(cli, "fvid")
252         fake = SimpleNamespace(
253             success=True, height=1080, path=str(tmp_path / "videos" / "fvid.mp4"),

```

46. user (2026-06-10T05:39:20.894Z)

```

099cf67 Manual-ingestion: normalize-annotations guard + positioning doc (#13)
0e4ccc7 Merge pull request #12 from avidullu/feat/collector-report-refinements
b8d40c8 feat(reporting): delivery-report debuggability refinements + faq-resolution golden test
94fec51 Merge pull request #11 from avidullu/feat/observability-reports
038424d feat(reporting): per-video observability report (collector)
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
2882a03 feat(ingest): validate-delivery ? per-video QA gate + structured report
23f2ed0 Merge pull request #8 from avidullu/feat/last-shot-outcome
M data/sports_metadata.db

```

```

{
  "commercial_only": true,
  "statuses": [
    "curated",
    "staging"
  ],
  "total_videos": 24,
  "total_segments": 1915,
  "eval_sets": [
    {
      "video_id": "my_custom_local_match",
      "sport": "badminton",
      "csv": "badminton/my_custom_local_match.csv",
      "local_path": "data/videos/my_custom_match.mp4",
      "video_url": null,
      "match_name": "My Custom Local Match",
      "fps": 30.0,
      "resolution": "1080p",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 2
    },
    {
      "video_id": "shuttleaset_21",
      "sport": "badminton",
      "csv": "badminton/shuttleaset_21.csv",
      "local_path": "./data/videos\\shuttleaset_21.mp4",
      "video_url": "https://www.youtube.com/watch?v=gloiZ_gTJaE",
      "match_name": "An Se Young Ratchanok Intanon YONEX Thailand Open 2021 QuarterFinals",
      "fps": 30.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 75
    },
    {
      "video_id": "shuttleaset_1",
      "sport": "badminton",

```

```

---
data/eval_export:
badminton
manifest.json

data/videos:
_diag.mp4.part
_diag.mp4.part-Frag320.part
_diag.mp4.ytdl
_t.mp4.part
_t.mp4.part-Frag447.part
_t.mp4.ytdl
_v.mp4.part
_v.mp4.part-Frag726.part
_v.mp4.ytdl
shuttleset_1.mp4
shuttleset_10.mp4
shuttleset_11.mp4
shuttleset_13.mp4
shuttleset_14.mp4
shuttleset_15.mp4
shuttleset_16.mp4
shuttleset_17.mp4
shuttleset_18.mp4
shuttleset_19.mp4
shuttleset_2.mp4
shuttleset_20.mp4
shuttleset_21.mp4
shuttleset_22.mp4
shuttleset_23.mp4
shuttleset_24.mp4
shuttleset_25.mp4
shuttleset_3.mp4
shuttleset_4.mp4
shuttleset_6.mp4
shuttleset_8.mp4
shuttleset_9.mp4

```

47. user (2026-06-10T05:39:50.793Z)

```

master
data/sports_metadata.db | Bin 184320 -> 200704 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
----- #

@pytest.mark.parametrize("argv,method", [
    ("prog", "status", "status"),
    ("prog", "curate", "curate"),
    ("prog", "export", "export_eval_set"),
    ("prog", "fetch", "fetch"),
])
def test_main_dispatch(argv, method):
    from src.cli import curate as curate_mod
    with mock.patch.object(curate_mod, "SportsDataCLI") as M, mock.patch("sys.argv", argv):
        curate_mod.main()
        getattr(M.return_value, method).assert_called_once()

def test_main_import_local_dispatch():
    from src.cli import curate as curate_mod
    argv = ["prog", "import-local", "--sport", "badminton", "--video-path", "v.mp4", "--
annotations-path", "a.json"]
    with mock.patch.object(curate_mod, "SportsDataCLI") as M, mock.patch("sys.argv", argv):
        curate_mod.main()

```

```
M.return_value.import_local.assert_called_once()
```

```
def test_main_no_command_prints_help():
    from src.cli import curate as curate_mod
    with mock.patch.object(curate_mod, "SportsDataCLI"), mock.patch("sys.argv", ["prog"]):
        curate_mod.main() # falls through to print_help()
```

48. user (2026-06-10T05:40:09.352Z)

```
2026-06-09 18:05:00.989597200 +0530 200704
Wed Jun 10 11:10:05 IST 2026
0d27836b263ea337971d78782ce7b1e1572f78b6 *data/sports_metadata.db
..... [100%]
124 passed in 2.38s
0d27836b263ea337971d78782ce7b1e1572f78b6 *data/sports_metadata.db
```

49. user (2026-06-10T05:40:30.263Z)

```
1     import os
2     import urllib.request
3     import urllib.parse
4     import csv
5     from typing import Dict, Any, List, Tuple
6     from tabulate import tabulate
7
8     from src.core.config import load_config
9     from src.core.db import SportsDatabase
10    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
11    from src.core.schemas.badminton import BadmintonRally
12    from src.parsers.badminton.coachai import CoachAIBadmintonParser
13    from src.core.validator import DatasetValidator
14    from src.core.downloader import download_best_video
15
16    class ShuttleSetCrawler:
17        MATCH_CSV_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set/match.csv"
18        RAW_BASE_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set"
19
20        def __init__(self, config_path: str = "config/config.yaml"):
21            self.config_path = config_path
22            self.config = self._load_config()
23            self.db_path = self.config.get("database_path", "data/sports_metadata.db")
24            self.db = SportsDatabase(self.db_path)
25            self.validator = DatasetValidator(config_path)
26
27            # Load storage config
28            storage = self.config.get("video_storage", {})
29            self.local_dir = storage.get("local_dir", "./data/videos")
30            self.max_download_size_mb = storage.get("max_download_size_mb", 1000)
31            self.auto_download = storage.get("auto_download", True)
32            # Optional resolution cap (e.g. 1080 to avoid pulling 4K). Unset = best
available.
33            self.max_height = storage.get("max_height")
34
35            # Load ingestion limits
36            limits = self.config.get("ingestion_limits", {})
37            self.limit_max_total_size_mb = limits.get("max_total_download_size_mb", 5000)
38            self.limit_max_video_count = limits.get("max_total_video_count", 25)
39
40            os.makedirs(self.local_dir, exist_ok=True)
41            os.makedirs("data/temp", exist_ok=True)
42
43        def _load_config(self) -> Dict[str, Any]:
```

```

44         return load_config(self.config_path)
45
46     def fetch_and_curate(self):
47         """Downloads ShuttleSet metadata, crawls match videos, parses rallies, and
enforces limits."""
48         print("Starting ShuttleSet dataset ingestion...")
49         print(f"Ingestion limits: Max Videos={self.limit_max_video_count}, Max Total
Size={self.limit_max_size_mb_str()}")
50
51         # 1. Download match.csv metadata
52         temp_match_csv = "data/temp/shuttleaset_match.csv"
53         try:
54             print(f"Downloading match catalog from {self.MATCH_CSV_URL}...")
55             urllib.request.urlretrieve(self.MATCH_CSV_URL, temp_match_csv)
56         except Exception as e:
57             print(f"Error fetching match list: {e}")
58             return
59
60         # Read matches
61         matches: List[Dict[str, Any]] = []
62         with open(temp_match_csv, mode='r', encoding='utf-8') as f:
63             reader = csv.DictReader(f)
64             for row in reader:
65                 matches.append(row)
66
67         # 2. Iterate through matches
68         videos_downloaded = 0
69         total_size_downloaded_mb = 0.0
70         total_rallies_indexed = 0
71         db_videos_added = 0
72         db_videos_updated = 0
73
74         # Detailed summary logs
75         summary_log: List[Dict[str, Any]] = []
76
77         print(f"Found {len(matches)} matches in ShuttleSet catalog.")
78
79         for idx, match in enumerate(matches):
80             # Check limits
81             if videos_downloaded >= self.limit_max_video_count:
82                 print(f"\nLimit reached: Maximum video count
({self.limit_max_video_count}) reached. Stopping ingestion.")
83                 break
84             if total_size_downloaded_mb >= self.limit_max_total_size_mb:
85                 print(f"\nLimit reached: Maximum download size
({self.limit_max_total_size_mb} MB) reached. Stopping.")
86                 break
87
88             video_dir_name = match.get("video")
89             youtube_url = match.get("url")
90             match_id = match.get("id")
91
92             # Skip catalog rows missing the fields we depend on rather than
93             # crashing the entire ingestion run.
94             if not video_dir_name or not match_id:
95                 print(f"\n[{idx+1}/{len(matches)}] Skipping catalog row with missing
'video'/'id' column.")
96                 continue
97
98             try:
99                 set_count = int(float(str(match.get("set", "2")).strip() or "2"))
100             except ValueError:
101                 set_count = 2
102             match_name = video_dir_name.replace("_", " ")
103             video_id = f"shuttleaset_{match_id}"

```

```

104
105 # ShuttleSet is MIT licensed -> commercially friendly
106 license_type = LicenseType.MIT
107 is_commercial = True
108
109 print(f"\n[{idx+1}/{len(matches)}] Processing: {match_name}")
110 print(f"  YouTube URL: {youtube_url}")
111
112 # Download video files (if configured)
113 local_path = None
114 download_size_mb = 0.0
115
116 probed_resolution = None
117 probed_fps = None
118 if self.auto_download and youtube_url:
119     # 2.1 Download the best-resolution stream available (MP4). The shared
120     # downloader sorts formats by resolution and records the actual
121     # width/height/fps it obtained ? we never assume a resolution.
122     print("  Downloading best-resolution video...")
123     video_filename = f"{video_id}.mp4"
124     dest_path = os.path.join(self.local_dir, video_filename)
125
126     result = download_best_video(
127         youtube_url, dest_path,
128         max_height=self.max_height,
129         max_size_mb=self.max_download_size_mb,
130     )
131     if result.success:
132         local_path = result.path
133         download_size_mb = result.size_mb
134         probed_resolution = result.resolution_label
135         probed_fps = result.fps
136         videos_downloaded += 1
137         total_size_downloaded_mb += download_size_mb
138         print(f"  Successfully downloaded video "
139               f"({download_size_mb:.2f} MB, {probed_resolution or 'unknown'
140               f'({f'{', {probed_fps:g}fps' if probed_fps else ''})")
141     else:
142         print(f"  Failed to download video: {result.error}")
143         local_path = None
144         download_size_mb = 0.0
145
146 # 2.2 Download & Merge stroke CSV files for this match
147 print("  Fetching annotation files...")
148 rallies_merged: List[BadmintonRally] = []
149
150 # Temporary storage to merge set csvs
151 merged_csv_path = f"data/temp/{video_id}_merged.csv"
152 headers = ["set", "rally", "ball_round", "time", "type", "score_a",
153 "score_b"]
154
155 with open(merged_csv_path, 'w', newline='', encoding='utf-8') as mf:
156     writer = csv.writer(mf)
157     writer.writerow(headers)
158
159     for s in range(1, set_count + 1):
160         encoded_dir = urllib.parse.quote(video_dir_name)
161         set_url = f"{self.RAW_BASE_URL}/{encoded_dir}/set{s}.csv"
162         set_temp_path = f"data/temp/{video_id}_set{s}.csv"
163         try:
164             urllib.request.urlretrieve(set_url, set_temp_path)
165
166             # Read and write rows, adding set number
167             with open(set_temp_path, 'r', encoding='utf-8') as sf:

```

```

167         sreader = csv.DictReader(sf)
168         # Normalize fieldnames
169         sreader.fieldnames = [n.lower().strip() for n in
sreader.fieldnames] if sreader.fieldnames else []
170
171         for row in sreader:
172             # Write to merged file
173             writer.writerow([
174                 s,
175                 row.get("rally", "0"),
176                 row.get("ball_round", row.get("ball_seq", "1")),
177                 row.get("time", row.get("timestamp", "0.0")),
178                 row.get("type", ""),
179                 row.get("score_a", row.get("round_score_a", "0")),
180                 row.get("score_b", row.get("round_score_b", "0"))
181             ])
182         # Clean up set CSV
183         os.remove(set_temp_path)
184     except Exception as e:
185         print(f" Warning: Failed to fetch Set {s} CSV: {e}")
186
187 # 2.3 Parse merged annotations
188 rallies = []
189 video_meta = None
190 if os.path.exists(merged_csv_path):
191     parser = CoachAIBadmintonParser(start_buffer=1.0, end_buffer=2.0)
192     try:
193         video_meta, rallies = parser.parse(
194             merged_csv_path,
195             video_id=video_id,
196             video_url=youtube_url,
197             license=license_type.value,
198             match_name=match_name,
199             local_path=local_path
200         )
201     # Clean up merged CSV
202     os.remove(merged_csv_path)
203 except Exception as e:
204     print(f" Error parsing merged CSV: {e}")
205     rallies = []
206
207 # Override the parser's placeholder resolution/fps with what we actually
208 # downloaded, so the DB reflects the real file (the indexer validates
209 # resolution and keys by it).
210 if video_meta is not None:
211     if probed_resolution:
212         video_meta.resolution = probed_resolution
213     if probed_fps:
214         video_meta.fps = probed_fps
215
216 if not video_meta or not rallies:
217     print(" Skipping match due to missing metadata or rallies.")
218     if local_path and os.path.exists(local_path):
219         os.remove(local_path)
220     # Roll back download accounting for the now-discarded file so
221     # the limit checks and summary stay accurate.
222     videos_downloaded -= 1
223     total_size_downloaded_mb -= download_size_mb
224     continue
225
226 # 2.4 Validate rallies
227 is_valid, errors = self.validator.validate_badminton_rallies(rallies)
228 if not is_valid:
229     print(f" Warning: Rallies failed validation. Skipping DB insertion.")
230     for err in errors:

```

```

231         print(f"    - {err}")
232     if local_path and os.path.exists(local_path):
233         os.remove(local_path)
234         # Roll back download accounting for the now-discarded file.
235         videos_downloaded -= 1
236         total_size_downloaded_mb -= download_size_mb
237     continue
238
239     # 2.5 Insert to Database
240     # Update metrics
241     existing = self.db.get_video(video_id)
242     if existing:
243         db_videos_updated += 1
244     else:
245         db_videos_added += 1
246
247     self.db.insert_video(video_meta)
248     count = self.db.insert_badminton_rallies(rallies)
249     total_rallies_indexed += count
250
251     print(f"    Successfully indexed {count} rallies in SQLite database.")
252
253     summary_log.append({
254         "match_name": match_name,
255         "video_id": video_id,
256         "downloaded": local_path is not None,
257         "size_mb": download_size_mb,
258         "rallies": count
259     })
260
261     # 3. Print execution summary
262     self._print_execution_summary(
263         videos_downloaded,
264         total_size_downloaded_mb,
265         total_rallies_indexed,
266         db_videos_added,
267         db_videos_updated,
268         summary_log
269     )
270
271     # Cleanup temp directory (best-effort).
272     try:
273         for f in os.listdir("data/temp"):
274             os.remove(os.path.join("data/temp", f))
275         os.rmdir("data/temp")
276     except OSError:
277         pass
278
279     def limit_max_size_mb_str(self) -> str:
280         return f"{self.limit_max_total_size_mb} MB"
281
282     def _print_execution_summary(self,
283                                videos_downloaded: int,
284                                total_size_mb: float,
285                                total_rallies: int,
286                                added: int,
287                                updated: int,
288                                log: List[Dict[str, Any]]):
289         print("\n" + "="*50)
290         print("                INGESTION EXECUTION SUMMARY                ")
291         print("="*50)
292         print(f"Total Video Files Downloaded: {videos_downloaded}")
293         print(f"Total Download Size           : {total_size_mb:.2f} MB")
294         print(f"Total Rallies Indexed          : {total_rallies}")
295         print(f"Database Matches Added         : {added}")

```



```

296         print(f"Database Matches Updated      : {updated}")
297         print("-"*50)
298
299         if log:
300             print("Match Log Breakdown:")
301             table_data = []
302             for item in log:
303                 table_data.append([
304                     item["match_name"],
305                     item["video_id"],
306                     "Yes" if item["downloaded"] else "No",
307                     f"{item['size_mb']:.2f} MB",
308                     item["rallies"]
309                 ])
310             print(tabulate(table_data, headers=["Match Name", "Video ID", "Downloaded",
311 "File Size", "Rallies"], tablefmt="simple"))
312         else:
313             print("No matches were successfully processed.")
314             print("-"*50 + "\n")
315
316     def main():
317         crawler = ShuttleSetCrawler()
318         crawler.fetch_and_curate()
319
320     if __name__ == "__main__":
321         main()

```

50. user (2026-06-10T05:41:35.198Z)

```

.
..
.coverage
.git
.gitattributes
.gitignore
.pytest_cache
.sixth
README.md
__pycache__
cleanup.py
config
data
docs
pyproject.toml
src
tests
NO CI
12:     parser.add_argument("--license", default="MIT", help="Dataset license type")
31:         license=args.license,
42:         print(f" - License: {video_meta.license_type.value}")

```

51. assistant (2026-06-10T05:42:27.899Z)

All evidence gathered. Compiling the audit now.

52. user (2026-06-10T05:44:07.674Z)

Structured output provided successfully

53. assistant (2026-06-10T05:44:08.382Z)

You've hit your session limit · resets 3:50pm (Asia/Calcutta)